FreshAir Desk

Source Code Appendix

Firmware, sensor modules, dashboard, AI server, and launch scripts

This appendix lists the project source files in a print-friendly format. It includes firmware sketches and modules, the Python Flask AI server, setup and upload scripts, dependency requirements, and the example environment file.

Credential handling: The private .env file is excluded. Wi-Fi identifiers, passwords, API keys, tokens, and other credential values are replaced with [REDACTED] in this copy. Do not upload a PDF that contains live credentials.

Included source files: 21

Source File Index

Project path	Lines
ai_server/.env.example	3
ai_server/requirements.txt	5
ai_server/server.py	210
ai_server/start.bat	74
Check_ESP32_Status.bat	38
firmware/00_hello/00_hello.ino	31
firmware/01_i2c_scanner/01_i2c_scanner.ino	51
firmware/01_sensor_proof/01_sensor_proof.ino	106
firmware/02_dht22_test/02_dht22_test.ino	43
firmware/02_mq135_calibration/02_mq135_calibration.ino	139
firmware/03_bh1750_test/03_bh1750_test.ino	49
firmware/03_dashboard/03_dashboard.ino	677
firmware/04_mq135_test/04_mq135_test.ino	61
firmware/05_integrated_monitor/05_integrated_monitor.ino	79
firmware/05_integrated_monitor/classify.cpp	79
firmware/05_integrated_monitor/classify.h	24
firmware/05_integrated_monitor/config.h	56
firmware/05_integrated_monitor/sensors.cpp	103
firmware/05_integrated_monitor/sensors.h	25
setup.bat	92
Upload_iPhone_Firmware.bat	2

ai_server/.env.example

3 source lines

```
0001  # Get your free key at: https://console.groq.com → API Keys
0002  # Copy this file to .env and paste your key below
0003  GROQ_API_KEY=[REDACTED]
```

ai_server/requirements.txt

5 source lines

```
0001  flask
0002  flask-cors
0003  python-dotenv
0004  requests
0005  groq
```

ai_server/server.py

210 source lines

```
0001  import os, sys, time, json, threading, subprocess, requests
0002  sys.stdout.reconfigure(encoding=&#x27;utf-8&#x27;, errors=&#x27;replace&#x27;)
0003  from datetime import datetime
0004  from flask import Flask, jsonify
0005  from flask_cors import CORS
0006  from dotenv import load_dotenv
0007  from groq import Groq
0008
0009  load_dotenv()
0010
0011  api_key = os.environ.get(&quot;GROQ_API_KEY&quot;)
0012  if not api_key:
0013      print(&quot;[ERROR] GROQ_API_KEY not set. Add it to ai_server/.env&quot;)
0014      exit(1)
0015
0016  client = Groq(api_key=api_key)
0017
0018  MODEL      = &quot;qwen/qwen3.8-27b&quot;
0019  ESP32_URLS = [&quot;http://172.20.10.5/json&quot;]
0020  POLL_EVERY = 30   # seconds between automatic refreshes (Groq free: 14400 req/day)
0021
0022  app = Flask(__name__)
0023  CORS(app)
0024
0025  cache = {
0026      &quot;summary&quot;:      &quot;Starting up — first reading in a moment...&quot;,
0027      &quot;suggestions&quot;:  [],
0028      &quot;score&quot;:        None,
0029      &quot;focus_impact&quot;: &quot;&quot;,
0030      &quot;updated&quot;:      0,
0031      &quot;error&quot;:        None
0032  }
0033  lock          = threading.Lock()
0034  last_overall  = None   # track last known status for change detection
0035
0036
0037  def now():
0038      return datetime.now().strftime(&quot;%H:%M:%S&quot;)
0039
0040
0041  def build_prompt(data):
0042      return f&quot;&quot;&quot;You are an indoor desk environment assistant. Analyse the readings below and respond with ONLY a JSON object — no
      markdown, no code fences.
0043
0044  Sensor readings right now:
0045  - Temperature : {data.get(&#x27;temp&#x27;,  &#x27;N/A&#x27;)} degC  (ideal 21-26 degC)
0046  - Humidity    : {data.get(&#x27;hum&#x27;,   &#x27;N/A&#x27;)} %     (ideal 40-60 %)
0047  - Light       : {data.get(&#x27;lux&#x27;,   &#x27;N/A&#x27;)} lux   (ideal 6 lux; &lt;4 = too dark, &gt;8 = too bright)
0048  - Air quality : {data.get(&#x27;ratio&#x27;, &#x27;N/A&#x27;)} RS/R0 (clean air ~3.6; lower = more pollutants)
0049  - Overall     : {data.get(&#x27;overall&#x27;, &#x27;N/A&#x27;)}
0050
0051  Return exactly this JSON:
0052  {{
0053    &quot;summary&quot;: &quot;&lt;2-3 sentences about current environment&gt;&quot;,
```

```
  &amp;quot;suggestions&amp;quot;: [&amp;quot;&amp;lt;action 1&amp;gt;&amp;quot;, &amp;quot;&amp;lt;action 2&amp;gt;&amp;quot;],
  &amp;quot;score&amp;quot;: &amp;lt;integer 1-10&amp;gt;,
  &amp;quot;focus_impact&amp;quot;: &amp;quot;&amp;lt;one sentence on how this affects focus/productivity&amp;gt;&amp;quot;
}}&amp;quot;&amp;quot;&amp;quot;


def parse_model_json(text):
    text = text.strip()
    if &amp;quot;```&amp;quot; in text:
        for part in text.split(&amp;quot;```&amp;quot;):
            part = part.strip().lstrip(&amp;quot;json&amp;quot;).strip()
            try:
                return json.loads(part)
            except Exception:
                continue
    return json.loads(text)


def call_ai(data):
    prompt = build_prompt(data)
    for attempt in range(3):
        try:
            resp = client.chat.completions.create(
                model=MODEL,
                messages=[{&amp;quot;role&amp;quot;: &amp;quot;user&amp;quot;, &amp;quot;content&amp;quot;: prompt}],
                temperature=0.3,
                max_tokens=400,
            )
            parsed = parse_model_json(resp.choices[0].message.content)
            print(f&amp;quot;[{now()}] Groq OK (attempt {attempt+1})&amp;quot;, flush=True)
            return parsed
        except Exception as e:
            err_str = str(e)
            if &amp;quot;429&amp;quot; in err_str or &amp;quot;rate_limit&amp;quot; in err_str.lower():
                wait = 10 * (attempt + 1)
                print(f&amp;quot;[{now()}] Rate limited, waiting {wait}s...&amp;quot;, flush=True)
                time.sleep(wait)
            elif &amp;quot;503&amp;quot; in err_str or &amp;quot;502&amp;quot; in err_str:
                wait = 5 * (attempt + 1)
                print(f&amp;quot;[{now()}] Groq overloaded, waiting {wait}s...&amp;quot;, flush=True)
                time.sleep(wait)
            else:
                print(f&amp;quot;[{now()}] Groq error: {err_str[:120]}&amp;quot;, flush=True)
                raise
    raise Exception(&amp;quot;Groq failed after 3 attempts&amp;quot;)


def fetch_and_update():
    global last_overall

    # Reach ESP32
    data = None
    for url in ESP32_URLS:
        try:
            resp = requests.get(url, timeout=5)
            resp.raise_for_status()
            data = resp.json()
            break
        except Exception:
```

```
            continue
    if data is None:
        msg = &quot;ESP32 not reachable — check it is powered and on the same hotspot.&quot;
        print(f&quot;[{now()}] {msg}&quot;)
        with lock:
            cache[&quot;error&quot;] = msg
        return

    # Check if overall status changed — if so, force AI refresh regardless of timer
    current_overall = data.get(&quot;overall&quot;)
    changed = (current_overall != last_overall)
    if changed and last_overall is not None:
        print(f&quot;[{now()}] Status changed: {last_overall} -&gt; {current_overall}. Refreshing AI suggestion...&quot;)
    last_overall = current_overall

    # Request AI recommendations from Groq
    try:
        parsed = call_ai(data)
        parsed[&quot;updated&quot;] = int(time.time())
        parsed[&quot;error&quot;]   = None
        with lock:
            cache.update(parsed)
        print(f&quot;[{now()}] Suggestion updated. Score: {parsed.get(&#x27;score&#x27;)}/10 | {current_overall}&quot;)
    except Exception as e:
        print(f&quot;[{now()}] Groq recommendation request failed: {e}&quot;)
        with lock:
            cache[&quot;error&quot;] = &quot;AI temporarily unavailable — will retry shortly.&quot;


def background_poller():
    global last_overall
    while True:
        time.sleep(POLL_EVERY)
        # Also force a refresh if status changed (poll ESP32 quickly to detect change)
        try:
            for url in ESP32_URLS:
                try:
                    data = requests.get(url, timeout=4).json()
                    current = data.get(&quot;overall&quot;)
                    if current != last_overall:
                        fetch_and_update()
                        break
                    break
                except Exception:
                    continue
            else:
                fetch_and_update()
        except Exception:
            fetch_and_update()


@app.route(&quot;/suggest&quot;)
def get_suggestion():
    with lock:
        return jsonify(cache)


@app.route(&quot;/suggest/refresh&quot;, methods=[&quot;POST&quot;])
```

```
def refresh_now():
    # Fire in background — don&#x27;t block the HTTP response
    threading.Thread(target=fetch_and_update, daemon=True).start()
    with lock:
        resp = dict(cache)
    resp[&quot;refreshing&quot;] = True
    return jsonify(resp)


if __name__ == &quot;__main__&quot;:
    all_ips = []
    try:
        out = subprocess.check_output(&quot;ipconfig&quot;, shell=True).decode(errors=&quot;ignore&quot;)
        for line in out.splitlines():
            if &quot;IPv4&quot; in line:
                ip = line.split(&quot;:&quot;)[-1].strip()
                if not ip.startswith(&quot;127.&quot;):
                    all_ips.append(ip)
    except Exception:
        pass

    # First fetch immediately in background
    threading.Thread(target=fetch_and_update, daemon=True).start()
    threading.Thread(target=background_poller, daemon=True).start()

    print()
    print(&quot;=&quot; * 54)
    print(&quot;  FreshAir Desk -- AI Server&quot;)
    print(&quot;=&quot; * 54)
    print(&quot;  Detected IPs:&quot;)
    for ip in all_ips:
        label = &quot;&quot;
        if ip.startswith(&quot;172.20.10.&quot;):   label = &quot;  &lt;&lt; iPhone hotspot (AI server network)&quot;
        print(f&quot;    {ip}{label}&quot;)
    print()
    print(f&quot;  Model: {MODEL}  |  Poll every {POLL_EVERY}s&quot;)
    print(&quot;=&quot; * 54)
    print()

    app.run(host=&quot;0.0.0.0&quot;, port=5000, debug=False)
```

ai_server/start.bat

74 source lines

```
0001  @echo off
0002  setlocal EnableExtensions
0003  title FreshAir Desk AI Server
0004  cd /d &quot;%~dp0&quot;
0005
0006  echo.
0007  echo ================================================
0008  echo   FreshAir Desk - AI Server
0009  echo ================================================
0010  echo.
0011
0012  if not exist &quot;%~dp0.env&quot; goto noenv
0013
0014  set &quot;PYTHON=%USERPROFILE%\anaconda3\python.exe&quot;
0015  if exist &quot;%PYTHON%&quot; goto pythonfound
0016
0017  set &quot;PYTHON=%USERPROFILE%\miniconda3\python.exe&quot;
0018  if exist &quot;%PYTHON%&quot; goto pythonfound
0019
0020  set &quot;PYTHON=%LOCALAPPDATA%\Programs\Python\Python314\python.exe&quot;
0021  if exist &quot;%PYTHON%&quot; goto pythonfound
0022
0023  set &quot;PYTHON=%LOCALAPPDATA%\Programs\Python\Python313\python.exe&quot;
0024  if exist &quot;%PYTHON%&quot; goto pythonfound
0025
0026  set &quot;PYTHON=%LOCALAPPDATA%\Programs\Python\Python312\python.exe&quot;
0027  if exist &quot;%PYTHON%&quot; goto pythonfound
0028
0029  set &quot;PYTHON=%LOCALAPPDATA%\Programs\Python\Python311\python.exe&quot;
0030  if exist &quot;%PYTHON%&quot; goto pythonfound
0031
0032  echo [ERROR] Python was not found. Install Anaconda or Python, then try again.
0033  goto finish
0034
0035  :pythonfound
0036  echo [OK] Using Python:
0037  echo %PYTHON%
0038  &quot;%PYTHON%&quot; --version
0039  if errorlevel 1 goto pythonerror
0040
0041  echo.
0042  echo [SETUP] Installing required packages. Please leave this window open...
0043  &quot;%PYTHON%&quot; -m pip install -r &quot;%~dp0requirements.txt&quot;
0044  if errorlevel 1 goto piperror
0045
0046  echo.
0047  echo [OK] Packages are ready. Starting the AI server...
0048  echo Keep this window open while using FreshAir Desk.
0049  echo.
0050  &quot;%PYTHON%&quot; &quot;%~dp0server.py&quot;
0051  echo.
0052  echo The AI server has stopped.
0053  goto finish
0054
```

```
:noenv
echo [ERROR] The AI settings file is missing:
echo %~dp0.env
echo Contact me before changing or copying API keys.
goto finish

:pythonerror
echo.
echo [ERROR] Python could not start. The AI server did not start.
goto finish

:piperror
echo.
echo [ERROR] Required packages could not be installed. Check internet access.
goto finish

:finish
echo.
pause
endlocal
```

Check_ESP32_Status.bat

38 source lines

```
0001  @echo off
0002  setlocal EnableExtensions
0003  title FreshAir Desk - ESP32 Status
0004
0005  set &quot;ARDUINO_CLI_DIR=%LOCALAPPDATA%\Programs\Arduino IDE\resources\app\lib\backend\resources&quot;
0006  if not exist &quot;%ARDUINO_CLI_DIR%\arduino-cli.exe&quot; set &quot;ARDUINO_CLI_DIR=%LOCALAPPDATA%\Programs\arduino-ide\resources\app\lib\backend\
      resources&quot;
0007  if not exist &quot;%ARDUINO_CLI_DIR%\arduino-cli.exe&quot; (
0008      echo Arduino IDE command-line tools were not found.
0009      echo Open Arduino IDE once, then try again.
0010      goto :done
0011  )
0012  set &quot;ARDUINO_CLI=%ARDUINO_CLI_DIR%\arduino-cli.exe&quot;
0013  set &quot;PORT_LIST=%TEMP%\freshair_board_list.txt&quot;
0014  set &quot;PORT_MATCH=%TEMP%\freshair_board_port.txt&quot;
0015
0016  echo Finding the ESP32 USB connection...
0017  &quot;%ARDUINO_CLI%&quot; board list &gt; &quot;%PORT_LIST%&quot; 2&gt;&amp;1
0018  type &quot;%PORT_LIST%&quot;
0019  set &quot;ESP_PORT=&quot;
0020  findstr /R /I /C:&quot;^COM[0-9][0-9]*&quot; &quot;%PORT_LIST%&quot; &gt; &quot;%PORT_MATCH%&quot;
0021  for /f &quot;usebackq tokens=1&quot; %%P in (&quot;%PORT_MATCH%&quot;) do if not defined ESP_PORT set &quot;ESP_PORT=%%P&quot;
0022  if not defined ESP_PORT (
0023      echo.
0024      echo No COM port found. Connect the ESP32 to the laptop by USB and try again.
0025      goto :done
0026  )
0027
0028  echo.
0029  echo Opening the ESP32 status window on %ESP_PORT%.
0030  echo Press the ESP32 EN/RESET button once to show its startup messages.
0031  echo Leave the laptop connected to the iPhone hotspot.
0032  echo.
0033  &quot;%ARDUINO_CLI%&quot; monitor -p %ESP_PORT% --config baudrate=115200
0034
0035  :done
0036  echo.
0037  pause
0038  endlocal
```

firmware/00_hello/00_hello.ino

31 source lines

```
0001  // FreshAir Desk — ESP32 bring-up sketch
0002  // Verifies board boots, serial works, and uptime increments correctly.
0003
0004  #define BAUD_RATE     115200
0005  #define INTERVAL_MS   2000
0006
0007  unsigned long last_print = 0;
0008
0009  void setup() {
0010    Serial.begin(BAUD_RATE);
0011    delay(500);
0012    Serial.println(&amp;quot;=== FreshAir Desk ESP32 bring-up ===&amp;quot;);
0013    Serial.print(&amp;quot;Board: &amp;quot;);
0014    Serial.println(ARDUINO_BOARD);
0015    Serial.print(&amp;quot;CPU freq: &amp;quot;);
0016    Serial.print(ESP.getCpuFreqMHz());
0017    Serial.println(&amp;quot; MHz&amp;quot;);
0018    Serial.println(&amp;quot;Serial OK. Printing uptime every 2 s.&amp;quot;);
0019    Serial.println(&amp;quot;====================================&amp;quot;);
0020  }
0021
0022  void loop() {
0023    unsigned long now = millis();
0024    if (now - last_print &amp;gt;= INTERVAL_MS) {
0025      last_print = now;
0026      Serial.print(&amp;quot;uptime_ms=&amp;quot;);
0027      Serial.print(now);
0028      Serial.print(&amp;quot;  uptime_s=&amp;quot;);
0029      Serial.println(now / 1000);
0030    }
0031  }
```

firmware/01_i2c_scanner/01_i2c_scanner.ino

51 source lines

```
0001  // FreshAir Desk — I2C address scanner
0002  // Run this before trusting any BH1750 wiring/address in code.
0003  // project.md: &amp;quot;Confirm its I2C address with a scanner before integrating it.&amp;quot;
0004
0005  #include &amp;lt;Wire.h&amp;gt;
0006
0007  #define SDA_PIN 21
0008  #define SCL_PIN 22
0009  #define BAUD_RATE 115200
0010  #define SCAN_INTERVAL_MS 5000
0011
0012  unsigned long last_scan = 0;
0013
0014  void scan_bus() {
0015    Serial.println(&amp;quot;Scanning I2C bus...&amp;quot;);
0016    int found = 0;
0017    for (uint8_t address = 1; address &amp;lt; 127; address++) {
0018      Wire.beginTransmission(address);
0019      uint8_t error = Wire.endTransmission();
0020      if (error == 0) {
0021        Serial.print(&amp;quot;  device found at 0x&amp;quot;);
0022        if (address &amp;lt; 16) Serial.print(&amp;quot;0&amp;quot;);
0023        Serial.println(address, HEX);
0024        found++;
0025      }
0026    }
0027    if (found == 0) {
0028      Serial.println(&amp;quot;  no I2C devices found — check SDA/SCL wiring and GND.&amp;quot;);
0029    } else {
0030      Serial.print(&amp;quot;  total devices found: &amp;quot;);
0031      Serial.println(found);
0032    }
0033    Serial.println(&amp;quot;---&amp;quot;);
0034  }
0035
0036  void setup() {
0037    Serial.begin(BAUD_RATE);
0038    delay(500);
0039    Wire.begin(SDA_PIN, SCL_PIN);
0040    Serial.println(&amp;quot;=== FreshAir Desk I2C scanner ===&amp;quot;);
0041    Serial.print(&amp;quot;SDA=GPIO&amp;quot;); Serial.print(SDA_PIN);
0042    Serial.print(&amp;quot; SCL=GPIO&amp;quot;); Serial.println(SCL_PIN);
0043  }
0044
0045  void loop() {
0046    unsigned long now = millis();
0047    if (now - last_scan &amp;gt;= SCAN_INTERVAL_MS || last_scan == 0) {
0048      last_scan = now;
0049      scan_bus();
0050    }
0051  }
```

firmware/01_sensor_proof/01_sensor_proof.ino

106 source lines

```
0001  // FreshAir Desk — Milestone 3: sensors + LED status (R0-calibrated)
0002  // DHT22 OUT→19 | BH1750 SDA→21 SCL→22 | MQ135 A0→34 D0→32
0003  // LED RED→25   | LED GREEN→33
0004
0005  #include &lt;Wire.h&gt;
0006  #include &lt;BH1750.h&gt;
0007  #include &lt;DHT.h&gt;
0008
0009  #define DHT_PIN    19
0010  #define DHT_TYPE   DHT22
0011  #define MQ135_AOUT 34
0012  #define MQ135_DOUT 32
0013  #define LED_RED    25
0014  #define LED_GREEN  33
0015
0016  // MQ135 calibration — R0 measured after 20h warm-up in clean air
0017  #define MQ135_RL   10000.0   // load resistor on module (ohms)
0018  #define MQ135_VCC  3.3
0019  #define MQ135_R0   12741.0   // from calibration: RS in clean air / 3.6
0020
0021  // RS/R0 thresholds (clean air = 3.6; lower = more gas detected)
0022  #define MQ135_RATIO_SAFE  2.0   // below this = poor air trend
0023
0024  // Safe environment thresholds
0025  #define TEMP_MIN  18.0
0026  #define TEMP_MAX  30.0
0027  #define HUM_MIN   30.0
0028  #define HUM_MAX   65.0
0029
0030  DHT    dht(DHT_PIN, DHT_TYPE);
0031  BH1750 lightMeter;
0032
0033  float readMQ135Ratio() {
0034    int   raw = analogRead(MQ135_AOUT);
0035    float vrl = raw * (MQ135_VCC / 4095.0);
0036    if (vrl &lt; 0.001) return -1;
0037    float rs    = ((MQ135_VCC - vrl) / vrl) * MQ135_RL;
0038    return rs / MQ135_R0;
0039  }
0040
0041  void blink(int pin, int times = 2, int ms = 150) {
0042    for (int i = 0; i &lt; times; i++) {
0043      digitalWrite(pin, HIGH); delay(ms);
0044      digitalWrite(pin, LOW);  delay(ms);
0045    }
0046  }
0047
0048  void setup() {
0049    Serial.begin(115200);
0050    while (!Serial) {}
0051
0052    pinMode(LED_RED,   OUTPUT);
0053    pinMode(LED_GREEN, OUTPUT);
0054
```

```
  dht.begin();
  Wire.begin(21, 22);
  if (!lightMeter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE)) {
    Serial.println(&amp;quot;[BH1750] init failed&amp;quot;);
  }
  pinMode(MQ135_DOUT, INPUT);

  Serial.println(&amp;quot;uptime_s | temp_C | hum_pct | lux   | rs_r0 | status&amp;quot;);
  Serial.println(&amp;quot;-----------------------------------------------------------&amp;quot;);
}

void loop() {
  float temp  = dht.readTemperature();
  float hum   = dht.readHumidity();
  float lux   = lightMeter.readLightLevel();
  float ratio = readMQ135Ratio();

  bool dht_ok = !isnan(temp) &amp;amp;&amp;amp; !isnan(hum);
  bool bh_ok  = lux &amp;gt;= 0;
  bool mq_ok  = ratio &amp;gt; 0;

  bool safe = dht_ok &amp;amp;&amp;amp; bh_ok &amp;amp;&amp;amp; mq_ok
            &amp;amp;&amp;amp; temp  &amp;gt;= TEMP_MIN  &amp;amp;&amp;amp; temp  &amp;lt;= TEMP_MAX
            &amp;amp;&amp;amp; hum   &amp;gt;= HUM_MIN   &amp;amp;&amp;amp; hum   &amp;lt;= HUM_MAX
            &amp;amp;&amp;amp; ratio &amp;gt;= MQ135_RATIO_SAFE;

  // Serial output
  Serial.print(millis() / 1000); Serial.print(&amp;quot; | &amp;quot;);

  if (!dht_ok) Serial.print(&amp;quot;DHT_ERR | DHT_ERR&amp;quot;);
  else {
    Serial.print(temp, 1); Serial.print(&amp;quot; | &amp;quot;);
    Serial.print(hum,  1);
  }
  Serial.print(&amp;quot; | &amp;quot;);

  if (!bh_ok) Serial.print(&amp;quot;BH_ERR &amp;quot;);
  else        Serial.print(lux, 1);
  Serial.print(&amp;quot; | &amp;quot;);

  if (!mq_ok) Serial.print(&amp;quot;MQ_ERR&amp;quot;);
  else        Serial.print(ratio, 3);
  Serial.print(&amp;quot; | &amp;quot;);

  Serial.println(safe ? &amp;quot;SAFE&amp;quot; : &amp;quot;WARN&amp;quot;);

  // Blink the right LED
  digitalWrite(safe ? LED_RED   : LED_GREEN, LOW);
  blink(safe        ? LED_GREEN : LED_RED);

  delay(2000);
}
```

firmware/02_dht22_test/02_dht22_test.ino

43 source lines

```
0001  // FreshAir Desk — DHT22 standalone test
0002  // Requires: &quot;DHT sensor library&quot; by Adafruit + &quot;Adafruit Unified Sensor&quot;
0003  // Wiring: docs/diagrams/dht22_wiring.svg
0004
0005  #include &lt;DHT.h&gt;
0006
0007  #define DHT_PIN       4
0008  #define DHT_TYPE      DHT22
0009  #define BAUD_RATE     115200
0010  #define READ_INTERVAL_MS 2500   // DHT22 should not be polled faster than ~2s
0011
0012  DHT dht(DHT_PIN, DHT_TYPE);
0013  unsigned long last_read = 0;
0014
0015  void setup() {
0016    Serial.begin(BAUD_RATE);
0017    delay(500);
0018    dht.begin();
0019    Serial.println(&quot;=== FreshAir Desk DHT22 test ===&quot;);
0020    Serial.println(&quot;temperature_c,humidity_pct,valid&quot;);
0021  }
0022
0023  void loop() {
0024    unsigned long now = millis();
0025    if (now - last_read &lt; READ_INTERVAL_MS) {
0026      return;
0027    }
0028    last_read = now;
0029
0030    float humidity_pct = dht.readHumidity();
0031    float temperature_c = dht.readTemperature();
0032    bool valid = !isnan(humidity_pct) &amp;&amp; !isnan(temperature_c);
0033
0034    Serial.print(valid ? String(temperature_c, 1) : &quot;NaN&quot;);
0035    Serial.print(&quot;,&quot;);
0036    Serial.print(valid ? String(humidity_pct, 1) : &quot;NaN&quot;);
0037    Serial.print(&quot;,&quot;);
0038    Serial.println(valid ? &quot;true&quot; : &quot;false&quot;);
0039
0040    if (!valid) {
0041      Serial.println(&quot;  WARNING: read failed — check DATA pin and pull-up resistor.&quot;);
0042    }
0043  }
```

firmware/02_mq135_calibration/02_mq135_calibration.ino

139 source lines

```
0001  // FreshAir Desk — MQ135 R0 calibration
0002  // Run this ONCE in clean air (open window, no cooking/smoke/incense nearby)
0003  // after the sensor has been powered for at least 24h.
0004  // MQ135 A0 → GPIO34 | VCC → 3V3
0005
0006  #define MQ135_AOUT  34
0007  #define RL          10000.0   // load resistor on module in ohms (typically 10K)
0008  #define VCC         3.3       // supply voltage
0009  #define ADC_MAX     4095.0
0010  #define RS_R0_RATIO 3.6       // datasheet: RS/R0 = 3.6 in clean air
0011
0012  // Warm-up and sampling config
0013  #define WARMUP_MINUTES   5    // minimum before sampling; 24h ideal but 5m is usable
0014  #define SAMPLE_COUNT     200  // readings to average for R0
0015  #define SAMPLE_INTERVAL  2000 // ms between samples
0016
0017  float readRS() {
0018    int   raw = analogRead(MQ135_AOUT);
0019    float vrl = raw * (VCC / ADC_MAX);           // voltage across load resistor
0020    if (vrl &lt;= 0.001) return -1;                // guard: avoid divide-by-zero
0021    float rs  = ((VCC - vrl) / vrl) * RL;        // sensor resistance
0022    return rs;
0023  }
0024
0025  void warmup() {
0026    long warmup_ms = (long)WARMUP_MINUTES * 60 * 1000;
0027    long start     = millis();
0028    Serial.println(&quot;=== WARM-UP ===&quot;);
0029    Serial.print(&quot;Waiting &quot;); Serial.print(WARMUP_MINUTES); Serial.println(&quot; minutes...&quot;);
0030    Serial.println(&quot;(For best results power the sensor for 24h before running this sketch)&quot;);
0031    Serial.println();
0032
0033    while (millis() - start &lt; warmup_ms) {
0034      long remaining = (warmup_ms - (millis() - start)) / 1000;
0035      float rs = readRS();
0036      Serial.print(&quot;  remaining: &quot;); Serial.print(remaining);
0037      Serial.print(&quot;s  |  RS now: &quot;);
0038      if (rs &lt; 0) Serial.println(&quot;ERR (check wiring)&quot;);
0039      else { Serial.print(rs, 1); Serial.println(&quot; ohm&quot;); }
0040      delay(10000); // print every 10s during warm-up
0041    }
0042
0043    Serial.println();
0044    Serial.println(&quot;Warm-up done. Starting calibration samples...&quot;);
0045    Serial.println(&quot;------------------------------------------------&quot;);
0046  }
0047
0048  void setup() {
0049    Serial.begin(115200);
0050    while (!Serial) {}
0051
0052    Serial.println();
0053    Serial.println(&quot;========================================&quot;);
0054    Serial.println(&quot;  FreshAir Desk — MQ135 R0 Calibration&quot;);
```

```
0055    Serial.println(&quot;======================================&quot;);
0056    Serial.println(&quot;IMPORTANT: Run in clean outdoor-quality air.&quot;);
0057    Serial.println(&quot;  Open a window. No cooking, smoke, or&quot;);
0058    Serial.println(&quot;  incense in the room.&quot;);
0059    Serial.println();
0060    Serial.print(&quot;RL assumed: &quot;); Serial.print((int)RL); Serial.println(&quot; ohm&quot;);
0061    Serial.print(&quot;VCC:        &quot;); Serial.print(VCC);     Serial.println(&quot; V&quot;);
0062    Serial.println();
0063
0064    warmup();
0065  }
0066
0067  void loop() {
0068    double rs_sum  = 0;
0069    double rs_min  = 1e9;
0070    double rs_max  = 0;
0071    int    good    = 0;
0072    int    errors  = 0;
0073
0074    Serial.print(&quot;Collecting &quot;); Serial.print(SAMPLE_COUNT); Serial.println(&quot; samples...&quot;);
0075    Serial.println(&quot;  #   |   RS (ohm)&quot;);
0076    Serial.println(&quot;------+------------&quot;);
0077
0078    for (int i = 1; i &lt;= SAMPLE_COUNT; i++) {
0079      float rs = readRS();
0080
0081      if (rs &lt; 0) {
0082        errors++;
0083        Serial.print(&quot;  &quot;); Serial.print(i);
0084        Serial.println(&quot; |  ERR&quot;);
0085      } else {
0086        rs_sum += rs;
0087        if (rs &lt; rs_min) rs_min = rs;
0088        if (rs &gt; rs_max) rs_max = rs;
0089        good++;
0090
0091        Serial.print(&quot;  &quot;); Serial.print(i);
0092        Serial.print(&quot; |  &quot;); Serial.println(rs, 1);
0093      }
0094
0095      delay(SAMPLE_INTERVAL);
0096    }
0097
0098    // Results
0099    Serial.println();
0100    Serial.println(&quot;======================================&quot;);
0101    Serial.println(&quot;  CALIBRATION RESULTS&quot;);
0102    Serial.println(&quot;======================================&quot;);
0103
0104    if (good &lt; 10) {
0105      Serial.println(&quot;ERROR: too few valid samples. Check wiring.&quot;);
0106      Serial.println(&quot;Halting.&quot;);
0107      while (1) {}
0108    }
0109
0110    double rs_avg = rs_sum / good;
0111    double r0     = rs_avg / RS_R0_RATIO;
0112
0113    Serial.print(&quot;Samples:    &quot;); Serial.print(good);
```

```
0114    if (errors) { Serial.print(&quot;  (&quot;); Serial.print(errors); Serial.print(&quot; errors)&quot;); }
0115    Serial.println();
0116    Serial.print(&quot;RS average: &quot;); Serial.print(rs_avg, 1); Serial.println(&quot; ohm&quot;);
0117    Serial.print(&quot;RS min:     &quot;); Serial.print(rs_min, 1); Serial.println(&quot; ohm&quot;);
0118    Serial.print(&quot;RS max:     &quot;); Serial.print(rs_max, 1); Serial.println(&quot; ohm&quot;);
0119    Serial.println();
0120    Serial.print(&quot;&gt;&gt;&gt; R0 = &quot;); Serial.print(r0, 1); Serial.println(&quot; ohm &lt;&lt;&lt;&quot;);
0121    Serial.println();
0122    Serial.println(&quot;Copy R0 into your main sketch:&quot;);
0123    Serial.print(&quot;  #define MQ135_R0  &quot;); Serial.println(r0, 1);
0124    Serial.println();
0125
0126    // Spread check — warn if RS varied too much (sensor not settled or dirty air)
0127    double spread_pct = ((rs_max - rs_min) / rs_avg) * 100.0;
0128    if (spread_pct &gt; 20.0) {
0129      Serial.println(&quot;WARNING: RS spread &gt; 20%. Sensor may not be settled or&quot;);
0130      Serial.println(&quot;  air quality changed during sampling. Re-run after longer&quot;);
0131      Serial.println(&quot;  warm-up or in steadier conditions.&quot;);
0132    } else {
0133      Serial.println(&quot;Spread looks good. R0 is reliable.&quot;);
0134    }
0135
0136    Serial.println();
0137    Serial.println(&quot;Done. Reset the board to run again.&quot;);
0138    while (1) {}  // halt — results stay visible on Serial Monitor
0139  }
```

firmware/03_bh1750_test/03_bh1750_test.ino

49 source lines

```
0001  // FreshAir Desk — BH1750 (GY-302) standalone test
0002  // Requires: &amp;quot;BH1750&amp;quot; library by Christopher Laws (claws/BH1750)
0003  // Wiring: docs/diagrams/bh1750_wiring.svg
0004  // Run 01_i2c_scanner first to confirm the address below.
0005
0006  #include &amp;lt;Wire.h&amp;gt;
0007  #include &amp;lt;BH1750.h&amp;gt;
0008
0009  #define SDA_PIN 21
0010  #define SCL_PIN 22
0011  #define BH1750_ADDR 0x23   // change to 0x5C if ADDR is tied high
0012  #define BAUD_RATE 115200
0013  #define READ_INTERVAL_MS 1000
0014
0015  BH1750 light_meter(BH1750_ADDR);
0016  unsigned long last_read = 0;
0017
0018  void setup() {
0019    Serial.begin(BAUD_RATE);
0020    delay(500);
0021    Wire.begin(SDA_PIN, SCL_PIN);
0022
0023    if (light_meter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE)) {
0024      Serial.println(&amp;quot;=== FreshAir Desk BH1750 test ===&amp;quot;);
0025      Serial.println(&amp;quot;BH1750 init OK&amp;quot;);
0026    } else {
0027      Serial.println(&amp;quot;BH1750 init FAILED — check wiring/address (run 01_i2c_scanner).&amp;quot;);
0028    }
0029    Serial.println(&amp;quot;light_lux,valid&amp;quot;);
0030  }
0031
0032  void loop() {
0033    unsigned long now = millis();
0034    if (now - last_read &amp;lt; READ_INTERVAL_MS) {
0035      return;
0036    }
0037    last_read = now;
0038
0039    float lux = light_meter.readLightLevel();
0040    bool valid = lux &amp;gt;= 0;
0041
0042    Serial.print(valid ? String(lux, 1) : &amp;quot;NaN&amp;quot;);
0043    Serial.print(&amp;quot;,&amp;quot;);
0044    Serial.println(valid ? &amp;quot;true&amp;quot; : &amp;quot;false&amp;quot;);
0045
0046    if (!valid) {
0047      Serial.println(&amp;quot;  WARNING: read failed — check I2C wiring/address.&amp;quot;);
0048    }
0049  }
```

firmware/03_dashboard/03_dashboard.ino

677 source lines

```
0001  // FreshAir Desk — Milestone 5+6: rule-based classifier + web dashboard
0002  // DHT22 OUT→19 | BH1750 SDA→21 SCL→22 | MQ135 A0→34 D0→32
0003  // LED RED→25   | LED GREEN→33
0004
0005  #include &lt;Wire.h&gt;
0006  #include &lt;BH1750.h&gt;
0007  #include &lt;DHT.h&gt;
0008  #include &lt;WiFi.h&gt;
0009  #include &lt;WebServer.h&gt;
0010
0011  // ── Wi-Fi — iPhone hotspot only ──────────────────────────────────
0012  // Network 1: iPhone hotspot
0013  const char* SSID1     = &quot;[REDACTED]&quot;;
0014  const char* PASSWORD1 = &quot;[REDACTED]&quot;;
0015  IPAddress STATIC_IP1(172, 20, 10,   5);
0016  IPAddress GATEWAY1  (172, 20, 10,   1);
0017  IPAddress SUBNET1   (255, 255, 255, 0);
0018  IPAddress DNS1      (172, 20, 10,   1);
0019  const char* AI_URL1 = &quot;http://172.20.10.2:5000&quot;;
0020
0021
0022  // Set at runtime by connectWiFi()
0023  String g_ai_server_url = &quot;&quot;;
0024
0025  // ── Pins ─────────────────────────────────────────────────────────
0026  #define DHT_PIN    19
0027  #define DHT_TYPE   DHT22
0028  #define MQ135_AOUT 34
0029  #define MQ135_DOUT 32
0030  #define LED_RED    25
0031  #define LED_GREEN  33
0032
0033  // ── MQ135 calibration ────────────────────────────────────────────
0034  #define MQ135_RL   10000.0
0035  #define MQ135_VCC  3.3
0036  #define MQ135_R0   12741.0
0037
0038  // ── Error codes ──────────────────────────────────────────────────
0039  // Printed to Serial whenever the fault occurs. Each includes a fix hint.
0040  #define EC_DHT_TIMEOUT  &quot;[E001] DHT22 read timeout    — fix: OUT→GPIO19, 10K pull-up on data line, 3V3+GND supply.&quot;
0041  #define EC_BH_INIT      &quot;[E002] BH1750 I2C not found  — fix: SDA→GPIO21, SCL→GPIO22, 3V3+GND supply, address 0x23.&quot;
0042  #define EC_BH_READ      &quot;[E003] BH1750 read failed    — fix: sensor lost I2C comms. Power-cycle the board.&quot;
0043  #define EC_MQ_SAT       &quot;[E004] MQ135 voltage OOB     — fix: A0→GPIO34, power from 3V3 (not 5V), allow ≥1 min warm-up.&quot;
0044  #define EC_WIFI_TIMEOUT &quot;[E005] WiFi connect timeout  — fix: hotspot name \&quot;iPhone\&quot;, password correct, phone in range.&quot;
0045  #define EC_WIFI_LOST    &quot;[E006] WiFi connection dropped — reconnecting...&quot;
0046  #define EC_WIFI_FAIL    &quot;[E007] WiFi reconnect failed — dashboard offline, serial readings continue.&quot;
0047
0048  // ── Classifier levels ────────────────────────────────────────────
0049  enum Level { GOOD, ATTENTION, POOR, SENSOR_ERR };
0050
0051  // Thresholds
0052  #define TEMP_GOOD_LO   21.0
0053  #define TEMP_GOOD_HI   26.0
0054  #define TEMP_ATTN_LO   18.0
```

```
#define TEMP_ATTN_HI   30.0
#define HUM_GOOD_LO    40.0
#define HUM_GOOD_HI    60.0
#define HUM_ATTN_LO    30.0
#define HUM_ATTN_HI    65.0
#define LUX_DARK       4.0   // below this = dark (Poor)
#define LUX_DIM        6.0   // 4-5 = better lighting needed (Attention)
#define LUX_IDEAL      7.0   // 6 = good; 7-8 = bright (Attention)
#define LUX_BRIGHT     8.0   // above this = too bright (Poor)
#define AIR_GOOD       3.0
#define AIR_ATTN       2.0

// ── History ──────────────────────────────────────────────────────
#define HISTORY_SIZE 60
struct Reading {
  float temp, hum, lux, ratio;
  Level overall;
  unsigned long ts;
};
Reading history[HISTORY_SIZE];
int histHead = 0, histCount = 0;

// ── Globals ──────────────────────────────────────────────────────
DHT       dht(DHT_PIN, DHT_TYPE);
BH1750    lightMeter;
WebServer server(80);

float g_temp  = 0, g_hum = 0, g_lux = 0, g_ratio = 0;
bool  g_dht_ok = false, g_bh_ok = false, g_mq_ok = false;
bool  g_bh_init_ok = false;
bool  g_wifi_ok = false;

Level g_lvl_temp, g_lvl_hum, g_lvl_lux, g_lvl_air, g_overall;

const char* g_tip_temp = &amp;quot;&amp;quot;;
const char* g_tip_hum  = &amp;quot;&amp;quot;;
const char* g_tip_lux  = &amp;quot;&amp;quot;;
const char* g_tip_air  = &amp;quot;&amp;quot;;

unsigned long lastRead = 0, lastHistory = 0, lastWifiCheck = 0;

// ── Classifier ───────────────────────────────────────────────────
Level classifyTemp(float t) {
  if (t &amp;gt;= TEMP_GOOD_LO &amp;amp;&amp;amp; t &amp;lt;= TEMP_GOOD_HI) return GOOD;
  if (t &amp;gt;= TEMP_ATTN_LO &amp;amp;&amp;amp; t &amp;lt;= TEMP_ATTN_HI) return ATTENTION;
  return POOR;
}
Level classifyHum(float h) {
  if (h &amp;gt;= HUM_GOOD_LO &amp;amp;&amp;amp; h &amp;lt;= HUM_GOOD_HI) return GOOD;
  if (h &amp;gt;= HUM_ATTN_LO &amp;amp;&amp;amp; h &amp;lt;= HUM_ATTN_HI) return ATTENTION;
  return POOR;
}
Level classifyLux(float l) {
  if (l &amp;lt; LUX_DARK)   return POOR;        // &amp;lt; 4: dark
  if (l &amp;lt; LUX_DIM)    return ATTENTION;   // 4–5: better lighting needed
  if (l &amp;lt; LUX_IDEAL)  return GOOD;        // 6: ideal
  if (l &amp;lt;= LUX_BRIGHT) return ATTENTION;  // 7–8: bright
  return POOR;                            // &amp;gt; 8: too bright
}
```

```
Level classifyAir(float r) {
  if (r &gt;= AIR_GOOD)  return GOOD;
  if (r &gt;= AIR_ATTN)  return ATTENTION;
  return POOR;
}

Level worstOf(Level a, Level b) { return (a &gt; b) ? a : b; }

const char* tipTemp(Level l, float t) {
  if (l == GOOD)          return &quot;Comfortable temperature for desk work.&quot;;
  if (t &gt; TEMP_ATTN_HI)  return &quot;Too hot — open a window or turn on a fan.&quot;;
  if (t &lt; TEMP_ATTN_LO)  return &quot;Too cold — close windows and warm the room.&quot;;
  if (t &gt; TEMP_GOOD_HI)  return &quot;Slightly warm — consider some ventilation.&quot;;
  return &quot;Slightly cool — a warm drink or light layer may help.&quot;;
}
const char* tipHum(Level l, float h) {
  if (l == GOOD)          return &quot;Humidity is in the ideal comfort range.&quot;;
  if (h &gt; HUM_ATTN_HI)   return &quot;Very humid — open a window or use a dehumidifier.&quot;;
  if (h &lt; HUM_ATTN_LO)   return &quot;Very dry — place a water bowl near your desk.&quot;;
  if (h &gt; HUM_GOOD_HI)   return &quot;Slightly humid — a bit of ventilation will help.&quot;;
  return &quot;Slightly dry — stay hydrated and consider a humidifier.&quot;;
}
const char* tipLux(Level l, float lux) {
  if (l == GOOD)      return &quot;Good lighting for focused desk work.&quot;;
  if (l == ATTENTION) return lux &lt; LUX_DIM
                           ? &quot;Better lighting needed — turn on your desk lamp.&quot;
                           : &quot;Bright — consider dimming the lights slightly.&quot;;
  return lux &gt; LUX_BRIGHT
         ? &quot;Too bright — close the blinds or dim overhead lights.&quot;
         : &quot;Too dark — switch on your desk lamp.&quot;;
}
const char* tipAir(Level l) {
  if (l == GOOD)      return &quot;Air quality looks clean and fresh.&quot;;
  if (l == ATTENTION) return &quot;Air getting stale — open a window soon.&quot;;
  return &quot;Poor air — open a window or step outside briefly.&quot;;
}

// ── LED feedback ──────────────────────────────────────────────
void blink(int pin, int times, int ms) {
  for (int i = 0; i &lt; times; i++) {
    digitalWrite(pin, HIGH); delay(ms);
    digitalWrite(pin, LOW);  delay(ms);
  }
}
void updateLED() {
  digitalWrite(LED_RED,   LOW);
  digitalWrite(LED_GREEN, LOW);
  if      (g_overall == GOOD)      blink(LED_GREEN, 2, 120);
  else if (g_overall == ATTENTION) blink(LED_RED,   2, 300);
  else                             blink(LED_RED,   4, 100);
}

// ── Sensor read + classify ────────────────────────────────────
void readAndClassify() {
  g_temp = dht.readTemperature();
  g_hum  = dht.readHumidity();
  g_dht_ok = !isnan(g_temp) &amp;&amp; !isnan(g_hum);
  if (!g_dht_ok) Serial.println(EC_DHT_TIMEOUT);
```

```
0172
0173    if (g_bh_init_ok) {
0174      g_lux   = lightMeter.readLightLevel();
0175      g_bh_ok = g_lux &gt;= 0;
0176      if (!g_bh_ok) Serial.println(EC_BH_READ);
0177    } else {
0178      g_bh_ok = false;
0179    }
0180
0181    int   raw = analogRead(MQ135_AOUT);
0182    float vrl = raw * (MQ135_VCC / 4095.0);
0183    g_mq_ok   = (vrl &gt; 0.001 &amp;&amp; vrl &lt; MQ135_VCC - 0.01);
0184    if (g_mq_ok) {
0185      g_ratio = ((MQ135_VCC - vrl) / vrl) * MQ135_RL / MQ135_R0;
0186    } else {
0187      g_ratio = -1;
0188      Serial.println(EC_MQ_SAT);
0189    }
0190
0191    g_lvl_temp = g_dht_ok ? classifyTemp(g_temp)  : SENSOR_ERR;
0192    g_lvl_hum  = g_dht_ok ? classifyHum(g_hum)    : SENSOR_ERR;
0193    g_lvl_lux  = g_bh_ok  ? classifyLux(g_lux)    : SENSOR_ERR;
0194    g_lvl_air  = g_mq_ok  ? classifyAir(g_ratio)  : SENSOR_ERR;
0195
0196    g_tip_temp = g_dht_ok ? tipTemp(g_lvl_temp, g_temp) : &quot;E001 — check DHT22 wiring on GPIO19.&quot;;
0197    g_tip_hum  = g_dht_ok ? tipHum(g_lvl_hum,  g_hum)  : &quot;E001 — check DHT22 wiring on GPIO19.&quot;;
0198    g_tip_lux  = g_bh_ok  ? tipLux(g_lvl_lux, g_lux)    : &quot;E002/E003 — check BH1750 on GPIO21/22.&quot;;
0199    g_tip_air  = g_mq_ok  ? tipAir(g_lvl_air)           : &quot;E004 — check MQ135 on GPIO34, 3V3 supply.&quot;;
0200
0201    g_overall = GOOD;
0202    if (g_lvl_temp != SENSOR_ERR) g_overall = worstOf(g_overall, g_lvl_temp);
0203    if (g_lvl_hum  != SENSOR_ERR) g_overall = worstOf(g_overall, g_lvl_hum);
0204    if (g_lvl_lux  != SENSOR_ERR) g_overall = worstOf(g_overall, g_lvl_lux);
0205    if (g_lvl_air  != SENSOR_ERR) g_overall = worstOf(g_overall, g_lvl_air);
0206    if ((g_lvl_temp == SENSOR_ERR || g_lvl_hum == SENSOR_ERR ||
0207         g_lvl_lux  == SENSOR_ERR || g_lvl_air  == SENSOR_ERR) &amp;&amp; g_overall == GOOD)
0208      g_overall = ATTENTION;
0209
0210    updateLED();
0211  }
0212
0213  void pushHistory() {
0214    Reading r;
0215    r.temp    = g_dht_ok ? g_temp  : -99;
0216    r.hum     = g_dht_ok ? g_hum   : -99;
0217    r.lux     = g_bh_ok  ? g_lux   : -1;
0218    r.ratio   = g_mq_ok  ? g_ratio : -1;
0219    r.overall = g_overall;
0220    r.ts      = millis() / 1000;
0221    history[histHead] = r;
0222    histHead = (histHead + 1) % HISTORY_SIZE;
0223    if (histCount &lt; HISTORY_SIZE) histCount++;
0224  }
0225
0226  // ── HTML helpers ──────────────────────────────────────────────
0227  String levelClass(Level l) {
0228    if (l == GOOD)      return &quot;good&quot;;
0229    if (l == ATTENTION) return &quot;attn&quot;;
0230    return &quot;poor&quot;;
```

```
}
String levelLabel(Level l) {
  if (l == GOOD)      return &quot;Good&quot;;
  if (l == ATTENTION) return &quot;Attention&quot;;
  if (l == POOR)      return &quot;Poor&quot;;
  return &quot;Error&quot;;
}
String overallBadge() {
  if (g_overall == GOOD)
    return &quot;&lt;span class=&#x27;badge badge-good&#x27;&gt;&amp;#10003;&amp;nbsp; All Good&lt;/span&gt;&quot;;
  if (g_overall == ATTENTION)
    return &quot;&lt;span class=&#x27;badge badge-attn&#x27;&gt;&amp;#9888;&amp;nbsp; Attention Needed&lt;/span&gt;&quot;;
  return &quot;&lt;span class=&#x27;badge badge-poor&#x27;&gt;&amp;#9888;&amp;nbsp; Poor Conditions&lt;/span&gt;&quot;;
}
String histPill(Level l) {
  if (l == GOOD)
    return &quot;&lt;span class=&#x27;pill pill-good&#x27;&gt;&lt;span class=&#x27;dot dot-good&#x27;&gt;&lt;/span&gt;Good&lt;/span&gt;&quot;;
  if (l == ATTENTION)
    return &quot;&lt;span class=&#x27;pill pill-attn&#x27;&gt;&lt;span class=&#x27;dot dot-attn&#x27;&gt;&lt;/span&gt;Attention&lt;/span&gt;&quot;;
  return &quot;&lt;span class=&#x27;pill pill-poor&#x27;&gt;&lt;span class=&#x27;dot dot-poor&#x27;&gt;&lt;/span&gt;Poor&lt;/span&gt;&quot;;
}

// ── Dashboard HTML ──────────────────────────────────────────
void handleRoot() {
  String html = R&quot;rawhtml(&lt;!DOCTYPE html&gt;
&lt;html lang=&quot;en&quot;&gt;
&lt;head&gt;
&lt;meta charset=&quot;UTF-8&quot;&gt;
&lt;meta name=&quot;viewport&quot; content=&quot;width=device-width,initial-scale=1&quot;&gt;
&lt;meta http-equiv=&quot;refresh&quot; content=&quot;5&quot;&gt;
&lt;title&gt;FreshAir Desk&lt;/title&gt;
&lt;style&gt;
*{box-sizing:border-box;margin:0;padding:0}
body{font-family:-apple-system,BlinkMacSystemFont,&#x27;Segoe UI&#x27;,sans-serif;background:#f0f4f8;color:#1a202c;min-height:100vh}
.topbar{background:#fff;border-bottom:1px solid #e2e8f0;padding:16px 24px;display:flex;align-items:center;justify-content:space-between;flex-wrap:
wrap;gap:10px}
.topbar-title{font-size:1.1rem;font-weight:700;color:#1a202c}
.topbar-sub{font-size:.73rem;color:#94a3b8;margin-top:2px}
.badge{padding:6px 16px;border-radius:999px;font-size:.78rem;font-weight:600;letter-spacing:.3px}
.badge-good{background:#dcfce7;color:#16a34a}
.badge-attn{background:#fef9c3;color:#a16207}
.badge-poor{background:#fee2e2;color:#dc2626}
.main{max-width:960px;margin:0 auto;padding:24px 16px}
.cards{display:grid;grid-template-columns:repeat(auto-fit,minmax(210px,1fr));gap:14px;margin-bottom:28px}
.card{background:#fff;border-radius:16px;padding:22px 18px 18px;box-shadow:0 1px 3px rgba(0,0,0,.06);border:1px solid #e2e8f0;position:relative;
overflow:hidden}
.card::before{content:&#x27;&#x27;;position:absolute;top:0;left:0;right:0;height:4px;border-radius:16px 16px 0 0}
.card-good::before{background:linear-gradient(90deg,#34d399,#10b981)}
.card-attn::before{background:linear-gradient(90deg,#fcd34d,#f59e0b)}
.card-poor::before{background:linear-gradient(90deg,#f87171,#ef4444)}
.card-icon{font-size:1.4rem;margin-bottom:8px}
.card-label{font-size:.68rem;font-weight:700;text-transform:uppercase;letter-spacing:1.2px;color:#94a3b8;margin-bottom:4px}
.card-value{font-size:2.4rem;font-weight:800;line-height:1;margin-bottom:2px}
.val-good{color:#059669}.val-attn{color:#d97706}.val-poor{color:#dc2626}.val-err{color:#94a3b8}
.card-unit{font-size:.75rem;color:#94a3b8;margin-bottom:10px}
.card-level{display:inline-block;font-size:.68rem;font-weight:700;padding:2px 9px;border-radius:999px;margin-bottom:8px;letter-spacing:.3px}
.lv-good{background:#dcfce7;color:#16a34a}
.lv-attn{background:#fef9c3;color:#a16207}
.lv-poor{background:#fee2e2;color:#dc2626}
```

```
.lv-err{background:#f1f5f9;color:#64748b}
.err-code{font-size:.68rem;font-family:monospace;color:#dc2626;font-weight:700;margin-bottom:4px}
.card-tip{font-size:.74rem;color:#64748b;line-height:1.45;border-top:1px solid #f1f5f9;padding-top:8px;margin-top:2px}
.section-title{font-size:.72rem;font-weight:700;text-transform:uppercase;letter-spacing:1px;color:#94a3b8;margin-bottom:10px}
.table-wrap{background:#fff;border-radius:16px;box-shadow:0 1px 3px rgba(0,0,0,.06);border:1px solid #e2e8f0;overflow:hidden}
table{width:100%;border-collapse:collapse;font-size:.81rem}
thead{background:#f8fafc}
th{text-align:left;padding:10px 14px;color:#64748b;font-weight:600;font-size:.7rem;text-transform:uppercase;letter-spacing:.5px;border-bottom:1px solid #e8edf3}
td{padding:9px 14px;border-bottom:1px solid #f1f5f9;color:#374151}
tr:last-child td{border-bottom:none}
tr:hover td{background:#fafbfc}
.pill{display:inline-flex;align-items:center;gap:4px;padding:2px 9px;border-radius:999px;font-size:.7rem;font-weight:600}
.pill-good{background:#dcfce7;color:#16a34a}
.pill-attn{background:#fef9c3;color:#a16207}
.pill-poor{background:#fee2e2;color:#dc2626}
.dot{width:6px;height:6px;border-radius:50%;display:inline-block;flex-shrink:0}
.dot-good{background:#22c55e}.dot-attn{background:#eab308}.dot-poor{background:#ef4444}
footer{text-align:center;font-size:.7rem;color:#94a3b8;margin-top:18px;padding-bottom:24px}
.ai-wrap{background:#fff;border-radius:16px;padding:20px 22px;box-shadow:0 1px 3px rgba(0,0,0,.06);border:1px solid #e2e8f0;margin-bottom:28px;border-top:4px solid #6366f1}
.ai-header{display:flex;align-items:center;justify-content:space-between;margin-bottom:14px;flex-wrap:wrap;gap:8px}
.ai-title{font-size:.7rem;font-weight:700;text-transform:uppercase;letter-spacing:1.2px;color:#6366f1}
.ai-score{width:50px;height:50px;border-radius:50%;display:flex;align-items:center;justify-content:center;font-size:1.3rem;font-weight:800;flex-shrink:0}
.sc-good{background:#dcfce7;color:#16a34a}.sc-attn{background:#fef9c3;color:#a16207}.sc-poor{background:#fee2e2;color:#dc2626}.sc-nil{background:#f1f5f9;color:#94a3b8}
.ai-summary{font-size:.85rem;color:#374151;line-height:1.65;margin-bottom:12px}
.ai-list{list-style:none;margin-bottom:12px}
.ai-list li{font-size:.81rem;color:#4b5563;padding:3px 0 3px 20px;position:relative;line-height:1.45}
.ai-list li::before{content:&#x27;→&#x27;;position:absolute;left:0;color:#6366f1;font-weight:700}
.ai-focus{font-size:.77rem;color:#6366f1;font-style:italic;padding:7px 12px;background:#f0f0ff;border-radius:8px;margin-bottom:12px}
.ai-footer{display:flex;align-items:center;justify-content:space-between;flex-wrap:wrap;gap:8px}
.ai-updated{font-size:.67rem;color:#94a3b8}
.ai-btn{background:#6366f1;color:#fff;border:none;padding:6px 16px;border-radius:8px;font-size:.74rem;font-weight:600;cursor:pointer;transition:background .15s}
.ai-btn:hover{background:#4f46e5}.ai-btn:disabled{background:#a5b4fc;cursor:not-allowed}
.ai-offline{font-size:.8rem;color:#94a3b8;font-style:italic;padding:8px 0}
&lt;/style&gt;
&lt;/head&gt;
&lt;body&gt;
)rawhtml&quot;;

  html += &quot;&lt;div class=&#x27;topbar&#x27;&gt;&quot;;
  html += &quot;&lt;div&gt;&lt;div class=&#x27;topbar-title&#x27;&gt;&amp;#127807; FreshAir Desk&lt;/div&gt;&quot;
          &quot;&lt;div class=&#x27;topbar-sub&#x27;&gt;Refreshes every 5 s &amp;nbsp;&amp;bull;&amp;nbsp; Uptime: &quot;
          + String(millis()/1000) + &quot; s&lt;/div&gt;&lt;/div&gt;&quot;;
  html += overallBadge();
  html += &quot;&lt;/div&gt;&lt;div class=&#x27;main&#x27;&gt;&quot;;

  html += &quot;&lt;div class=&#x27;cards&#x27;&gt;&quot;;

  // Temperature
  {
    String lc = levelClass(g_lvl_temp);
    html += &quot;&lt;div class=&#x27;card card-&quot; + lc + &quot;&#x27;&gt;&quot;;
    html += &quot;&lt;div class=&#x27;card-icon&#x27;&gt;&amp;#127777;&lt;/div&gt;&quot;;
    html += &quot;&lt;div class=&#x27;card-label&#x27;&gt;Temperature&lt;/div&gt;&quot;;
```

```
0341      if (!g_dht_ok) {
0342        html += &quot;&lt;div class=&#x27;err-code&#x27;&gt;E001&lt;/div&gt;&quot;;
0343        html += &quot;&lt;div class=&#x27;card-value val-err&#x27;&gt;—&lt;/div&gt;&quot;;
0344        html += &quot;&lt;div class=&#x27;card-unit&#x27;&gt;°C&lt;/div&gt;&quot;;
0345        html += &quot;&lt;span class=&#x27;card-level lv-err&#x27;&gt;Sensor Error&lt;/span&gt;&quot;;
0346      } else {
0347        html += &quot;&lt;div class=&#x27;card-value val-&quot; + lc + &quot;&#x27;&gt;&quot; + String(g_temp,1) + &quot;&lt;/div&gt;&quot;;
0348        html += &quot;&lt;div class=&#x27;card-unit&#x27;&gt;°C &amp;nbsp;·&amp;nbsp; ideal 21–26&lt;/div&gt;&quot;;
0349        html += &quot;&lt;span class=&#x27;card-level lv-&quot; + lc + &quot;&#x27;&gt;&quot; + levelLabel(g_lvl_temp) + &quot;&lt;/span&gt;&quot;;
0350      }
0351      html += &quot;&lt;div class=&#x27;card-tip&#x27;&gt;&quot; + String(g_tip_temp) + &quot;&lt;/div&gt;&lt;/div&gt;&quot;;
0352    }
0353
0354    // Humidity
0355    {
0356      String lc = levelClass(g_lvl_hum);
0357      html += &quot;&lt;div class=&#x27;card card-&quot; + lc + &quot;&#x27;&gt;&quot;;
0358      html += &quot;&lt;div class=&#x27;card-icon&#x27;&gt;&amp;#128167;&lt;/div&gt;&quot;;
0359      html += &quot;&lt;div class=&#x27;card-label&#x27;&gt;Humidity&lt;/div&gt;&quot;;
0360      if (!g_dht_ok) {
0361        html += &quot;&lt;div class=&#x27;err-code&#x27;&gt;E001&lt;/div&gt;&quot;;
0362        html += &quot;&lt;div class=&#x27;card-value val-err&#x27;&gt;—&lt;/div&gt;&quot;;
0363        html += &quot;&lt;div class=&#x27;card-unit&#x27;&gt;% RH&lt;/div&gt;&quot;;
0364        html += &quot;&lt;span class=&#x27;card-level lv-err&#x27;&gt;Sensor Error&lt;/span&gt;&quot;;
0365      } else {
0366        html += &quot;&lt;div class=&#x27;card-value val-&quot; + lc + &quot;&#x27;&gt;&quot; + String(g_hum,1) + &quot;&lt;/div&gt;&quot;;
0367        html += &quot;&lt;div class=&#x27;card-unit&#x27;&gt;% RH &amp;nbsp;·&amp;nbsp; ideal 40–60&lt;/div&gt;&quot;;
0368        html += &quot;&lt;span class=&#x27;card-level lv-&quot; + lc + &quot;&#x27;&gt;&quot; + levelLabel(g_lvl_hum) + &quot;&lt;/span&gt;&quot;;
0369      }
0370      html += &quot;&lt;div class=&#x27;card-tip&#x27;&gt;&quot; + String(g_tip_hum) + &quot;&lt;/div&gt;&lt;/div&gt;&quot;;
0371    }
0372
0373    // Light
0374    {
0375      String lc = levelClass(g_lvl_lux);
0376      html += &quot;&lt;div class=&#x27;card card-&quot; + lc + &quot;&#x27;&gt;&quot;;
0377      html += &quot;&lt;div class=&#x27;card-icon&#x27;&gt;&amp;#128161;&lt;/div&gt;&quot;;
0378      html += &quot;&lt;div class=&#x27;card-label&#x27;&gt;Light&lt;/div&gt;&quot;;
0379      if (!g_bh_ok) {
0380        html += &quot;&lt;div class=&#x27;err-code&#x27;&gt;&quot; + String(g_bh_init_ok ? &quot;E003&quot; : &quot;E002&quot;) + &quot;&lt;/div&gt;
      &quot;;
0381        html += &quot;&lt;div class=&#x27;card-value val-err&#x27;&gt;—&lt;/div&gt;&quot;;
0382        html += &quot;&lt;div class=&#x27;card-unit&#x27;&gt;lux&lt;/div&gt;&quot;;
0383        html += &quot;&lt;span class=&#x27;card-level lv-err&#x27;&gt;Sensor Error&lt;/span&gt;&quot;;
0384      } else {
0385        html += &quot;&lt;div class=&#x27;card-value val-&quot; + lc + &quot;&#x27;&gt;&quot; + String(g_lux,0) + &quot;&lt;/div&gt;&quot;;
0386        html += &quot;&lt;div class=&#x27;card-unit&#x27;&gt;lux &amp;nbsp;·&amp;nbsp; desk work &amp;gt;300&lt;/div&gt;&quot;;
0387        html += &quot;&lt;span class=&#x27;card-level lv-&quot; + lc + &quot;&#x27;&gt;&quot; + levelLabel(g_lvl_lux) + &quot;&lt;/span&gt;&quot;;
0388      }
0389      html += &quot;&lt;div class=&#x27;card-tip&#x27;&gt;&quot; + String(g_tip_lux) + &quot;&lt;/div&gt;&lt;/div&gt;&quot;;
0390    }
0391
0392    // Air Quality
0393    {
0394      String lc = levelClass(g_lvl_air);
0395      html += &quot;&lt;div class=&#x27;card card-&quot; + lc + &quot;&#x27;&gt;&quot;;
0396      html += &quot;&lt;div class=&#x27;card-icon&#x27;&gt;&amp;#129481;&lt;/div&gt;&quot;;
0397      html += &quot;&lt;div class=&#x27;card-label&#x27;&gt;Air Quality&lt;/div&gt;&quot;;
0398      if (!g_mq_ok) {
```

```
0399        html += &quot;&lt;div class=&#x27;err-code&#x27;&gt;E004&lt;/div&gt;&quot;;
0400        html += &quot;&lt;div class=&#x27;card-value val-err&#x27;&gt;—&lt;/div&gt;&quot;;
0401        html += &quot;&lt;div class=&#x27;card-unit&#x27;&gt;RS/R0&lt;/div&gt;&quot;;
0402        html += &quot;&lt;span class=&#x27;card-level lv-err&#x27;&gt;Sensor Error&lt;/span&gt;&quot;;
0403      } else {
0404        html += &quot;&lt;div class=&#x27;card-value val-&quot; + lc + &quot;&#x27;&gt;&quot; + String(g_ratio,2) + &quot;&lt;/div&gt;&quot;;
0405        html += &quot;&lt;div class=&#x27;card-unit&#x27;&gt;RS/R0 &amp;nbsp;·&amp;nbsp; clean air ~3.6&lt;/div&gt;&quot;;
0406        html += &quot;&lt;span class=&#x27;card-level lv-&quot; + lc + &quot;&#x27;&gt;&quot; + levelLabel(g_lvl_air) + &quot;&lt;/span&gt;&quot;;
0407      }
0408      html += &quot;&lt;div class=&#x27;card-tip&#x27;&gt;&quot; + String(g_tip_air) + &quot;&lt;/div&gt;&lt;/div&gt;&quot;;
0409    }
0410
0411    html += &quot;&lt;/div&gt;&quot;; // end cards
0412
0413    // ── AI Insights card ──────────────────────────────────────
0414    html += &quot;&lt;div class=&#x27;ai-wrap&#x27;&gt;&quot;;
0415    html += &quot;&lt;div class=&#x27;ai-header&#x27;&gt;&quot;;
0416    html += &quot;&lt;span class=&#x27;ai-title&#x27;&gt;&amp;#129504; AI Insights&lt;/span&gt;&quot;;
0417    html += &quot;&lt;div style=&#x27;display:flex;align-items:center;gap:12px&#x27;&gt;&quot;;
0418    html += &quot;&lt;div class=&#x27;ai-score sc-nil&#x27; id=&#x27;ai-score&#x27;&gt;—&lt;/div&gt;&quot;;
0419    html += &quot;&lt;button class=&#x27;ai-btn&#x27; id=&#x27;ai-btn&#x27; onclick=&#x27;refreshNow()&#x27;&gt;Refresh Now&lt;/button&gt;&quot;;
0420    html += &quot;&lt;/div&gt;&lt;/div&gt;&quot;;
0421    html += &quot;&lt;div id=&#x27;ai-body&#x27; style=&#x27;display:none&#x27;&gt;&quot;;
0422    html += &quot;&lt;div class=&#x27;ai-summary&#x27; id=&#x27;ai-summary&#x27;&gt;&lt;/div&gt;&quot;;
0423    html += &quot;&lt;ul class=&#x27;ai-list&#x27; id=&#x27;ai-list&#x27;&gt;&lt;/ul&gt;&quot;;
0424    html += &quot;&lt;div class=&#x27;ai-focus&#x27; id=&#x27;ai-focus&#x27; style=&#x27;display:none&#x27;&gt;&lt;/div&gt;&quot;;
0425    html += &quot;&lt;/div&gt;&quot;;
0426    html += &quot;&lt;div class=&#x27;ai-offline&#x27; id=&#x27;ai-offline&#x27; style=&#x27;display:none&#x27;&gt;AI server offline — run ai_server/
      start.bat on your laptop.&lt;/div&gt;&quot;;
0427    html += &quot;&lt;div class=&#x27;ai-footer&#x27;&gt;&quot;;
0428    html += &quot;&lt;span class=&#x27;ai-updated&#x27; id=&#x27;ai-updated&#x27;&gt;Waiting for first reading...&lt;/span&gt;&quot;;
0429    html += &quot;&lt;/div&gt;&lt;/div&gt;&quot;;
0430
0431    if (histCount &gt; 0) {
0432      html += &quot;&lt;div class=&#x27;section-title&#x27;&gt;History &amp;mdash; last 30 min&lt;/div&gt;&quot;;
0433      html += &quot;&lt;div class=&#x27;table-wrap&#x27;&gt;&lt;table&gt;&lt;thead&gt;&lt;tr&gt;&quot;
0434              &quot;&lt;th&gt;Time (s)&lt;/th&gt;&lt;th&gt;Temp °C&lt;/th&gt;&lt;th&gt;Hum %&lt;/th&gt;&quot;
0435              &quot;&lt;th&gt;Lux&lt;/th&gt;&lt;th&gt;RS/R0&lt;/th&gt;&lt;th&gt;Status&lt;/th&gt;&quot;
0436              &quot;&lt;/tr&gt;&lt;/thead&gt;&lt;tbody&gt;&quot;;
0437      for (int i = 0; i &lt; histCount; i++) {
0438        int idx = ((histHead - 1 - i) + HISTORY_SIZE) % HISTORY_SIZE;
0439        Reading&amp; r = history[idx];
0440        html += &quot;&lt;tr&gt;&lt;td&gt;&quot; + String(r.ts) + &quot;&lt;/td&gt;&quot;;
0441        html += &quot;&lt;td&gt;&quot; + (r.temp  == -99 ? &quot;&lt;span style=&#x27;color:#dc2626;font-family:monospace&#x27;&gt;E001&lt;/span&gt;
      &quot; : String(r.temp,  1)) + &quot;&lt;/td&gt;&quot;;
0442        html += &quot;&lt;td&gt;&quot; + (r.hum   == -99 ? &quot;&lt;span style=&#x27;color:#dc2626;font-family:monospace&#x27;&gt;E001&lt;/span&gt;
      &quot; : String(r.hum,   1)) + &quot;&lt;/td&gt;&quot;;
0443        html += &quot;&lt;td&gt;&quot; + (r.lux   == -1  ? &quot;&lt;span style=&#x27;color:#dc2626;font-family:monospace&#x27;&gt;E002&lt;/span&gt;
      &quot; : String(r.lux,   0)) + &quot;&lt;/td&gt;&quot;;
0444        html += &quot;&lt;td&gt;&quot; + (r.ratio == -1  ? &quot;&lt;span style=&#x27;color:#dc2626;font-family:monospace&#x27;&gt;E004&lt;/span&gt;
      &quot; : String(r.ratio, 2)) + &quot;&lt;/td&gt;&quot;;
0445        html += &quot;&lt;td&gt;&quot; + histPill(r.overall) + &quot;&lt;/td&gt;&lt;/tr&gt;&quot;;
0446      }
0447      html += &quot;&lt;/tbody&gt;&lt;/table&gt;&lt;/div&gt;&quot;;
0448    }
0449
0450    html += &quot;&lt;footer&gt;FreshAir Desk v0.7 &amp;nbsp;&amp;bull;&amp;nbsp; ESP32 @ &quot;;
0451    html += g_wifi_ok ? WiFi.localIP().toString() : &quot;offline&quot;;
0452    html += &quot;&lt;/footer&gt;&quot;;
```

```
0453
0454    // The laptop can receive different client addresses from the iPhone hotspot.
0455    // Probe its usable client range so the dashboard can find the local AI server.
0456    html += &quot;&lt;script&gt;const AI_URLS=Array.from({length:13},(_,i)=&gt;&#x27;http://172.20.10.&#x27;+(i+2)+&#x27;:5000&#x27;);&lt;/script&gt;
      &quot;;
0457    html += R&quot;rawjs(
0458  &lt;script&gt;
0459  let AI_URL=null;
0460  async function findServer(){
0461    const results=await Promise.all(AI_URLS.map(async u=&gt;{
0462      try{
0463        const r=await fetch(u+&#x27;/suggest&#x27;,{signal:AbortSignal.timeout(1000)});
0464        return r.ok?u:null;
0465      }catch(e){return null;}
0466    }));
0467    AI_URL=results.find(Boolean)||null;
0468    return AI_URL;
0469  }
0470  function scoreClass(n){return n&gt;=7?&#x27;sc-good&#x27;:n&gt;=4?&#x27;sc-attn&#x27;:&#x27;sc-poor&#x27;;}
0471  function renderSuggestion(d){
0472    const sc=d.score;
0473    const el=document.getElementById(&#x27;ai-score&#x27;);
0474    el.textContent=sc!=null?sc:&#x27;—&#x27;;
0475    el.className=&#x27;ai-score &#x27;+(sc!=null?scoreClass(sc):&#x27;sc-nil&#x27;);
0476    document.getElementById(&#x27;ai-summary&#x27;).textContent=d.summary||&#x27;—&#x27;;
0477    document.getElementById(&#x27;ai-list&#x27;).innerHTML=(d.suggestions||[]).map(s=&gt;&#x27;&lt;li&gt;&#x27;+s+&#x27;&lt;/li&gt;&#x27;).join(&#x27;
      &#x27;);
0478    const fi=document.getElementById(&#x27;ai-focus&#x27;);
0479    if(d.focus_impact){fi.textContent=d.focus_impact;fi.style.display=&#x27;block&#x27;;}else{fi.style.display=&#x27;none&#x27;;}
0480    document.getElementById(&#x27;ai-body&#x27;).style.display=&#x27;block&#x27;;
0481    document.getElementById(&#x27;ai-offline&#x27;).style.display=&#x27;none&#x27;;
0482    if(d.updated)document.getElementById(&#x27;ai-updated&#x27;).textContent=&#x27;Updated &#x27;+new Date(d.updated*1000).toLocaleTimeString();
0483  }
0484  async function fetchSuggestion(){
0485    const url=AI_URL||(await findServer());
0486    if(!url){
0487      document.getElementById(&#x27;ai-body&#x27;).style.display=&#x27;none&#x27;;
0488      document.getElementById(&#x27;ai-offline&#x27;).style.display=&#x27;block&#x27;;
0489      document.getElementById(&#x27;ai-updated&#x27;).textContent=&#x27;&#x27;;
0490      return;
0491    }
0492    try{
0493      const r=await fetch(url+&#x27;/suggest&#x27;,{signal:AbortSignal.timeout(6000)});
0494      renderSuggestion(await r.json());
0495    }catch(e){
0496      AI_URL=null; // force re-probe next time
0497      document.getElementById(&#x27;ai-body&#x27;).style.display=&#x27;none&#x27;;
0498      document.getElementById(&#x27;ai-offline&#x27;).style.display=&#x27;block&#x27;;
0499      document.getElementById(&#x27;ai-updated&#x27;).textContent=&#x27;&#x27;;
0500    }
0501  }
0502  async function refreshNow(){
0503    const btn=document.getElementById(&#x27;ai-btn&#x27;);
0504    btn.textContent=&#x27;Asking AI...&#x27;;btn.disabled=true;
0505    const url=AI_URL||(await findServer());
0506    if(!url){btn.textContent=&#x27;Refresh Now&#x27;;btn.disabled=false;return;}
0507    try{await fetch(url+&#x27;/suggest/refresh&#x27;,{method:&#x27;POST&#x27;,signal:AbortSignal.timeout(5000)});}catch(e){}
0508    let polls=20;
0509    const prev=document.getElementById(&#x27;ai-updated&#x27;).textContent;
```

```
  const id=setInterval(async()=&gt;{
    try{
      const r=await fetch(url+&#x27;/suggest&#x27;,{signal:AbortSignal.timeout(4000)});
      const d=await r.json();
      const ts=&#x27;Updated &#x27;+new Date(d.updated*1000).toLocaleTimeString();
      if(d.score!=null&amp;&amp;ts!==prev){renderSuggestion(d);clearInterval(id);btn.textContent=&#x27;Refresh Now&#x27;;btn.disabled=false;return;}
    }catch(e){}
    if(--polls&lt;=0){clearInterval(id);btn.textContent=&#x27;Refresh Now&#x27;;btn.disabled=false;}
  },3000);
}
fetchSuggestion();
setInterval(fetchSuggestion,30000);
&lt;/script&gt;
)rawjs&quot;;

  html += &quot;&lt;/div&gt;&lt;/body&gt;&lt;/html&gt;&quot;;
  server.send(200, &quot;text/html&quot;, html);
}

// ── JSON endpoint ──────────────────────────────────────────
void handleJson() {
  String json = &quot;{&quot;;
  json += &quot;\&quot;temp\&quot;:&quot;    + (g_dht_ok ? String(g_temp,  1) : String(&quot;null&quot;)) + &quot;,&quot;;
  json += &quot;\&quot;hum\&quot;:&quot;     + (g_dht_ok ? String(g_hum,   1) : String(&quot;null&quot;)) + &quot;,&quot;;
  json += &quot;\&quot;lux\&quot;:&quot;     + (g_bh_ok  ? String(g_lux,   1) : String(&quot;null&quot;)) + &quot;,&quot;;
  json += &quot;\&quot;ratio\&quot;:&quot;   + (g_mq_ok  ? String(g_ratio, 3) : String(&quot;null&quot;)) + &quot;,&quot;;
  json += &quot;\&quot;overall\&quot;:\&quot;&quot; + levelLabel(g_overall) + &quot;\&quot;,&quot;;
  json += &quot;\&quot;wifi\&quot;:&quot;    + String(g_wifi_ok ? &quot;true&quot; : &quot;false&quot;) + &quot;,&quot;;
  json += &quot;\&quot;uptime\&quot;:&quot;  + String(millis()/1000);
  json += &quot;}&quot;;
  server.send(200, &quot;application/json&quot;, json);
}

// ── WiFi connect helper ────────────────────────────────────
bool tryNetwork(const char* ssid, const char* pass,
                IPAddress sip, IPAddress gw, IPAddress sn, IPAddress dns,
                const char* aiUrl, int timeoutSec) {
  WiFi.disconnect(true);
  delay(300);
  WiFi.config(sip, gw, sn, dns);
  WiFi.begin(ssid, pass);
  Serial.print(&quot;[WIFI] Trying [&quot;); Serial.print(ssid); Serial.print(&quot;] &quot;);
  int tries = timeoutSec * 2;
  while (WiFi.status() != WL_CONNECTED &amp;&amp; tries-- &gt; 0) {
    delay(500); Serial.print(&quot;.&quot;);
  }
  if (WiFi.status() == WL_CONNECTED) {
    g_ai_server_url = String(aiUrl);
    Serial.println(&quot;\n[WIFI] Connected. IP: &quot; + WiFi.localIP().toString());
    Serial.println(&quot;[WIFI] AI server: &quot; + g_ai_server_url);
    return true;
  }
  Serial.println(&quot; failed.&quot;);
  return false;
}

bool connectWiFi() {
  WiFi.mode(WIFI_STA);
```

```
0568    Serial.println(&quot;[WIFI] Connecting to iPhone hotspot...&quot;);
0569    if (tryNetwork(SSID1, PASSWORD1, STATIC_IP1, GATEWAY1, SUBNET1, DNS1, AI_URL1, 30))
0570      return true;
0571    Serial.println(EC_WIFI_TIMEOUT);
0572    return false;
0573  }
0574
0575  // ── Setup ────────────────────────────────────────────────────────
0576  void setup() {
0577    Serial.begin(115200);
0578    delay(500);
0579
0580    Serial.println(&quot;\n====================================&quot;);
0581    Serial.println(&quot;  FreshAir Desk v0.6 — booting&quot;);
0582    Serial.println(&quot;====================================&quot;);
0583
0584    pinMode(LED_RED,   OUTPUT);
0585    pinMode(LED_GREEN, OUTPUT);
0586
0587    dht.begin();
0588    Serial.println(&quot;[INIT] DHT22 ready on GPIO19.&quot;);
0589
0590    Wire.begin(21, 22);
0591    g_bh_init_ok = lightMeter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE);
0592    if (g_bh_init_ok) {
0593      Serial.println(&quot;[INIT] BH1750 found on I2C (SDA=21, SCL=22).&quot;);
0594    } else {
0595      Serial.println(EC_BH_INIT);
0596    }
0597
0598    pinMode(MQ135_DOUT, INPUT);
0599    Serial.println(&quot;[INIT] MQ135 analog on GPIO34. Allow &gt;=1 min warm-up for stable readings.&quot;);
0600
0601    Serial.println(&quot;[INIT] Scanning Wi-Fi networks...&quot;);
0602    int n = WiFi.scanNetworks();
0603    for (int i = 0; i &lt; n; i++)
0604      Serial.println(&quot;  [&quot; + WiFi.SSID(i) + &quot;] ch=&quot; + String(WiFi.channel(i)) + &quot; rssi=&quot; + String(WiFi.RSSI(i)));
0605
0606    g_wifi_ok = connectWiFi();
0607    if (g_wifi_ok) {
0608      blink(LED_GREEN, 3, 200);
0609      server.on(&quot;/&quot;,     handleRoot);
0610      server.on(&quot;/json&quot;, handleJson);
0611      server.begin();
0612      Serial.println(&quot;[INIT] Web server started. Dashboard: http://&quot; + WiFi.localIP().toString());
0613    } else {
0614      blink(LED_RED, 5, 100);
0615      Serial.println(&quot;[INIT] Serial-only mode — no dashboard until WiFi connects.&quot;);
0616    }
0617
0618    readAndClassify();
0619    pushHistory();
0620    lastRead = lastHistory = lastWifiCheck = millis();
0621
0622    Serial.println();
0623    Serial.println(&quot;uptime_s | temp_C | hum_%  | lux    | rs_r0 | t_lvl     | h_lvl | l_lvl | a_lvl | overall&quot;);
0624    Serial.println(&quot;----------------------------------------------------------------------------------------------------&quot;);
0625  }
0626
```

```
// ── Loop ──────────────────────────────────────────────────────────
void loop() {
  if (g_wifi_ok) server.handleClient();

  unsigned long now = millis();

  // WiFi watchdog — check every 30 s, attempt reconnect if dropped
  if (now - lastWifiCheck &gt;= 30000) {
    lastWifiCheck = now;
    if (WiFi.status() != WL_CONNECTED) {
      Serial.println(EC_WIFI_LOST);
      WiFi.disconnect(true);
      delay(500);
      g_wifi_ok = connectWiFi();
      if (!g_wifi_ok) {
        Serial.println(EC_WIFI_FAIL);
      } else {
        server.on(&quot;/&quot;,     handleRoot);
        server.on(&quot;/json&quot;, handleJson);
        server.begin();
        Serial.println(&quot;[WIFI] Reconnected. Dashboard: http://&quot; + WiFi.localIP().toString());
      }
    }
  }

  if (now - lastRead &gt;= 2000) {
    lastRead = now;
    readAndClassify();

    Serial.print(now / 1000);
    Serial.print(&quot; | &quot;);
    Serial.print(g_dht_ok ? String(g_temp, 1) : &quot;E001  &quot;);
    Serial.print(&quot; | &quot;);
    Serial.print(g_dht_ok ? String(g_hum,  1) : &quot;E001  &quot;);
    Serial.print(&quot; | &quot;);
    Serial.print(g_bh_ok  ? String(g_lux,  1) : (g_bh_init_ok ? &quot;E003  &quot; : &quot;E002  &quot;));
    Serial.print(&quot; | &quot;);
    Serial.print(g_mq_ok  ? String(g_ratio, 3) : &quot;E004 &quot;);
    Serial.print(&quot; | &quot;);
    Serial.print(levelLabel(g_lvl_temp)); Serial.print(&quot; | &quot;);
    Serial.print(levelLabel(g_lvl_hum));  Serial.print(&quot; | &quot;);
    Serial.print(levelLabel(g_lvl_lux));  Serial.print(&quot; | &quot;);
    Serial.print(levelLabel(g_lvl_air));  Serial.print(&quot; | &quot;);
    Serial.println(levelLabel(g_overall));
  }

  if (now - lastHistory &gt;= 30000) {
    lastHistory = now;
    pushHistory();
  }
}
```

firmware/04_mq135_test/04_mq135_test.ino

61 source lines

```
0001  // FreshAir Desk — MQ135 standalone test
0002  // No library — MQ135 output is treated as a relative gas trend only, never a
0003  // calibrated CO2/eCO2/VOC/AQI value (rules.md, project.md).
0004  // Wiring: docs/diagrams/mq135_wiring.svg — powered from 3V3, read on GPIO34.
0005
0006  #define MQ135_PIN         34
0007  #define BAUD_RATE         115200
0008  #define READ_INTERVAL_MS  1000
0009  #define WARMUP_MS         (3UL * 60UL * 1000UL)  // 3 min minimum for a demo;
0010                                                     // real calibration needs far
0011                                                     // longer documented burn-in.
0012  #define SMOOTHING_SAMPLES 10
0013
0014  unsigned long last_read = 0;
0015  unsigned long boot_time = 0;
0016  int history[SMOOTHING_SAMPLES];
0017  int history_index = 0;
0018  bool history_filled = false;
0019
0020  void setup() {
0021    Serial.begin(BAUD_RATE);
0022    delay(500);
0023    analogReadResolution(12); // 0-4095 on ESP32
0024    boot_time = millis();
0025    Serial.println(&quot;=== FreshAir Desk MQ135 test ===&quot;);
0026    Serial.println(&quot;NOTE: gas_adc is a relative trend signal, not a calibrated&quot;);
0027    Serial.println(&quot;      CO2/eCO2/VOC/AQI reading.&quot;);
0028    Serial.println(&quot;gas_adc_raw,gas_adc_avg,warming_up&quot;);
0029  }
0030
0031  int smoothed_average() {
0032    int count = history_filled ? SMOOTHING_SAMPLES : history_index;
0033    if (count == 0) return 0;
0034    long sum = 0;
0035    for (int i = 0; i &lt; count; i++) sum += history[i];
0036    return sum / count;
0037  }
0038
0039  void loop() {
0040    unsigned long now = millis();
0041    if (now - last_read &lt; READ_INTERVAL_MS) {
0042      return;
0043    }
0044    last_read = now;
0045
0046    int raw = analogRead(MQ135_PIN);
0047    history[history_index] = raw;
0048    history_index++;
0049    if (history_index &gt;= SMOOTHING_SAMPLES) {
0050      history_index = 0;
0051      history_filled = true;
0052    }
0053
0054    bool warming_up = (now - boot_time) &lt; WARMUP_MS;
```

```
0055
0056    Serial.print(raw);
0057    Serial.print(&amp;quot;,&amp;quot;);
0058    Serial.print(smoothed_average());
0059    Serial.print(&amp;quot;,&amp;quot;);
0060    Serial.println(warming_up ? &amp;quot;true&amp;quot; : &amp;quot;false&amp;quot;);
0061  }
```

firmware/05_integrated_monitor/05_integrated_monitor.ino

79 source lines

```
0001  // FreshAir Desk — integrated monitor
0002  // Combines DHT22 + BH1750 + MQ135 on a fixed interval, validates readings,
0003  // classifies the desk environment with a documented rule set, and prints one
0004  // structured line per cycle plus one concrete recommendation.
0005  //
0006  // Requires: &quot;DHT sensor library&quot; + &quot;Adafruit Unified Sensor&quot; + &quot;BH1750&quot; (claws)
0007  // Wiring: docs/diagrams/full_system_wiring.svg
0008  // Run 01-04 individually first — see docs/SOP.md section 2.
0009
0010  #include &quot;config.h&quot;
0011  #include &quot;sensors.h&quot;
0012  #include &quot;classify.h&quot;
0013
0014  Reading reading;
0015  unsigned long last_sample = 0;
0016
0017  void setup_leds() {
0018    if (!LEDS_ENABLED) return;
0019    pinMode(PIN_LED_GOOD, OUTPUT);
0020    pinMode(PIN_LED_ATTENTION, OUTPUT);
0021    pinMode(PIN_LED_POOR, OUTPUT);
0022  }
0023
0024  void update_leds(Status overall) {
0025    if (!LEDS_ENABLED) return;
0026    digitalWrite(PIN_LED_GOOD, overall == STATUS_GOOD ? HIGH : LOW);
0027    digitalWrite(PIN_LED_ATTENTION, overall == STATUS_ATTENTION ? HIGH : LOW);
0028    digitalWrite(PIN_LED_POOR, overall == STATUS_POOR ? HIGH : LOW);
0029  }
0030
0031  void setup() {
0032    Serial.begin(BAUD_RATE);
0033    delay(500);
0034    Serial.println(&quot;=== FreshAir Desk integrated monitor ===&quot;);
0035    Serial.println(&quot;NOTE: gas readings are a relative trend, not calibrated CO2/eCO2/VOC/AQI.&quot;);
0036    Serial.println(&quot;uptime_s,temperature_c,humidity_pct,light_lux,gas_adc_avg,&quot;
0037                   &quot;comfort,light,gas,overall,recommendation&quot;);
0038
0039    sensors_init();
0040    setup_leds();
0041  }
0042
0043  void loop() {
0044    unsigned long now = millis();
0045    if (now - last_sample &lt; SAMPLE_INTERVAL_MS) {
0046      return; // millis()-based scheduling, no blocking delay() (rules.md)
0047    }
0048    last_sample = now;
0049
0050    sensors_read(reading);
0051
0052    Status comfort = classify_comfort(reading);
0053    Status light   = classify_light(reading);
0054    Status gas     = classify_gas(reading, sensors_gas_baseline());
```

```
  Status overall = overall_status(comfort, light, gas);
  String recommendation = build_recommendation(comfort, light, gas);

  update_leds(overall);

  Serial.print(now / 1000);
  Serial.print(&quot;,&quot;);
  Serial.print(reading.temp_valid ? String(reading.temperature_c, 1) : &quot;NaN&quot;);
  Serial.print(&quot;,&quot;);
  Serial.print(reading.humidity_valid ? String(reading.humidity_pct, 1) : &quot;NaN&quot;);
  Serial.print(&quot;,&quot;);
  Serial.print(reading.light_valid ? String(reading.light_lux, 1) : &quot;NaN&quot;);
  Serial.print(&quot;,&quot;);
  Serial.print(reading.gas_warming_up ? &quot;warming_up&quot; : String(reading.gas_adc_avg));
  Serial.print(&quot;,&quot;);
  Serial.print(status_name(comfort));
  Serial.print(&quot;,&quot;);
  Serial.print(status_name(light));
  Serial.print(&quot;,&quot;);
  Serial.print(status_name(gas));
  Serial.print(&quot;,&quot;);
  Serial.print(status_name(overall));
  Serial.print(&quot;,&quot;);
  Serial.println(recommendation);
}
```

firmware/05_integrated_monitor/classify.cpp

79 source lines

```
0001  // FreshAir Desk — rule-based environment classifier
0002  // Thresholds live in config.h so they can be revised after real testing.
0003  #include &amp;quot;classify.h&amp;quot;
0004  #include &amp;quot;config.h&amp;quot;
0005
0006  const char* status_name(Status s) {
0007    switch (s) {
0008      case STATUS_GOOD:      return &amp;quot;good&amp;quot;;
0009      case STATUS_ATTENTION: return &amp;quot;attention&amp;quot;;
0010      case STATUS_POOR:      return &amp;quot;poor&amp;quot;;
0011      default:                return &amp;quot;invalid&amp;quot;;
0012    }
0013  }
0014
0015  Status classify_comfort(const Reading &amp;amp;r) {
0016    if (!r.temp_valid || !r.humidity_valid) return STATUS_INVALID;
0017
0018    bool temp_good = (r.temperature_c &amp;gt;= TEMP_GOOD_MIN &amp;amp;&amp;amp; r.temperature_c &amp;lt;= TEMP_GOOD_MAX);
0019    bool temp_ok   = (r.temperature_c &amp;gt;= TEMP_OK_MIN   &amp;amp;&amp;amp; r.temperature_c &amp;lt;= TEMP_OK_MAX);
0020    bool hum_good  = (r.humidity_pct  &amp;gt;= HUM_GOOD_MIN  &amp;amp;&amp;amp; r.humidity_pct  &amp;lt;= HUM_GOOD_MAX);
0021    bool hum_ok    = (r.humidity_pct  &amp;gt;= HUM_OK_MIN    &amp;amp;&amp;amp; r.humidity_pct  &amp;lt;= HUM_OK_MAX);
0022
0023    if (temp_good &amp;amp;&amp;amp; hum_good) return STATUS_GOOD;
0024    if (temp_ok &amp;amp;&amp;amp; hum_ok)     return STATUS_ATTENTION;
0025    return STATUS_POOR;
0026  }
0027
0028  Status classify_light(const Reading &amp;amp;r) {
0029    if (!r.light_valid) return STATUS_INVALID;
0030
0031    if (r.light_lux &amp;gt;= LUX_GOOD_MIN &amp;amp;&amp;amp; r.light_lux &amp;lt;= LUX_GOOD_MAX) return STATUS_GOOD;
0032    if (r.light_lux &amp;gt;= LUX_OK_MIN   &amp;amp;&amp;amp; r.light_lux &amp;lt;= LUX_OK_MAX)   return STATUS_ATTENTION;
0033    return STATUS_POOR;
0034  }
0035
0036  Status classify_gas(const Reading &amp;amp;r, int baseline) {
0037    if (!r.gas_valid || r.gas_warming_up || baseline &amp;lt; 0) return STATUS_INVALID;
0038
0039    int delta = r.gas_adc_avg - baseline;
0040    if (delta &amp;lt;= GAS_ATTENTION_DELTA) return STATUS_GOOD;
0041    if (delta &amp;lt;= GAS_POOR_DELTA)      return STATUS_ATTENTION;
0042    return STATUS_POOR;
0043  }
0044
0045  Status overall_status(Status comfort, Status light, Status gas) {
0046    // Worst-of-three, ignoring dimensions that are still invalid/warming up so
0047    // one un-warmed sensor doesn&amp;#x27;t mask real problems elsewhere while also not
0048    // being falsely reported as &amp;quot;poor&amp;quot; itself.
0049    Status worst = STATUS_GOOD;
0050    Status values[3] = { comfort, light, gas };
0051    bool any_valid = false;
0052    for (int i = 0; i &amp;lt; 3; i++) {
0053      if (values[i] == STATUS_INVALID) continue;
0054      any_valid = true;
```

```
    if (values[i] &gt; worst) worst = values[i];
  }
  return any_valid ? worst : STATUS_INVALID;
}

String build_recommendation(Status comfort, Status light, Status gas) {
  String parts = &quot;&quot;;
  if (comfort == STATUS_POOR || comfort == STATUS_ATTENTION) {
    parts += &quot;Adjust room temperature/humidity (fan, heater, or ventilation). &quot;;
  }
  if (light == STATUS_POOR || light == STATUS_ATTENTION) {
    parts += &quot;Improve desk lighting. &quot;;
  }
  if (gas == STATUS_POOR || gas == STATUS_ATTENTION) {
    parts += &quot;Ventilate the room — open a window or improve airflow. &quot;;
  }
  if (parts.length() == 0) {
    if (comfort == STATUS_INVALID &amp;&amp; light == STATUS_INVALID &amp;&amp; gas == STATUS_INVALID) {
      parts = &quot;No valid readings yet.&quot;;
    } else {
      parts = &quot;Conditions look good — no action needed.&quot;;
    }
  }
  return parts;
}
```

firmware/05_integrated_monitor/classify.h

24 source lines

```
0001  // FreshAir Desk — rule-based environment classifier
0002  // This is a documented threshold rule set, not an AI/ML model (rules.md).
0003  #ifndef FRESHAIR_CLASSIFY_H
0004  #define FRESHAIR_CLASSIFY_H
0005
0006  #include &lt;Arduino.h&gt;
0007  #include &quot;sensors.h&quot;
0008
0009  enum Status {
0010    STATUS_INVALID = 0,
0011    STATUS_GOOD,
0012    STATUS_ATTENTION,
0013    STATUS_POOR
0014  };
0015
0016  const char* status_name(Status s);
0017
0018  Status classify_comfort(const Reading &amp;r);
0019  Status classify_light(const Reading &amp;r);
0020  Status classify_gas(const Reading &amp;r, int baseline);
0021  Status overall_status(Status comfort, Status light, Status gas);
0022  String build_recommendation(Status comfort, Status light, Status gas);
0023
0024  #endif
```

firmware/05_integrated_monitor/config.h

56 source lines

```
0001  // FreshAir Desk — integrated monitor configuration
0002  // All pins and thresholds live here so they can be revised after real
0003  // testing without hunting through the rest of the sketch (rules.md).
0004
0005  #ifndef FRESHAIR_CONFIG_H
0006  #define FRESHAIR_CONFIG_H
0007
0008  // ---- Pins (docs/pin_map.md) ----
0009  #define PIN_DHT22        4
0010  #define PIN_I2C_SDA      21
0011  #define PIN_I2C_SCL      22
0012  #define PIN_MQ135_AOUT   34
0013
0014  #define PIN_LED_GOOD      25
0015  #define PIN_LED_ATTENTION 26
0016  #define PIN_LED_POOR      27
0017  #define LEDS_ENABLED      false  // set true once LEDs are wired (see resistor warning in pin_map.md)
0018
0019  #define BH1750_ADDR 0x23   // confirm with 01_i2c_scanner, change to 0x5C if needed
0020
0021  // ---- Timing ----
0022  #define BAUD_RATE            115200
0023  #define SAMPLE_INTERVAL_MS   2500UL   // limited by DHT22&#x27;s max sample rate
0024  #define MQ135_WARMUP_MS       (3UL * 60UL * 1000UL)
0025  #define GAS_SMOOTHING_SAMPLES 10
0026
0027  // A raw ADC reading pinned at either rail (disconnected, shorted-to-ground,
0028  // or a saturated sensor) for longer than this is treated as a stuck/failed
0029  // input rather than a real gas trend.
0030  #define GAS_RAIL_LOW          5
0031  #define GAS_RAIL_HIGH         4090
0032  #define GAS_RAIL_TIMEOUT_MS   (30UL * 1000UL)
0033
0034  // ---- Comfort thresholds (temperature °C / humidity %) ----
0035  #define TEMP_GOOD_MIN   20.0
0036  #define TEMP_GOOD_MAX   26.0
0037  #define TEMP_OK_MIN     18.0
0038  #define TEMP_OK_MAX     29.0
0039  #define HUM_GOOD_MIN    30.0
0040  #define HUM_GOOD_MAX    60.0
0041  #define HUM_OK_MIN      20.0
0042  #define HUM_OK_MAX      70.0
0043
0044  // ---- Light thresholds (lux, desk task lighting) ----
0045  #define LUX_GOOD_MIN    300.0
0046  #define LUX_GOOD_MAX    750.0
0047  #define LUX_OK_MIN      150.0
0048  #define LUX_OK_MAX      1000.0
0049
0050  // ---- Gas trend thresholds (ADC counts above the boot-time baseline) ----
0051  // Placeholder values pending real calibration — see rules.md: MQ135 output is
0052  // a relative trend, not a calibrated concentration.
0053  #define GAS_ATTENTION_DELTA  150
0054  #define GAS_POOR_DELTA       400
```

```
0055
0056  #endif
```

firmware/05_integrated_monitor/sensors.cpp

103 source lines

```
0001  // FreshAir Desk — sensor acquisition, validation, and smoothing
0002  #include &amp;quot;sensors.h&amp;quot;
0003  #include &amp;quot;config.h&amp;quot;
0004  #include &amp;lt;DHT.h&amp;gt;
0005  #include &amp;lt;Wire.h&amp;gt;
0006  #include &amp;lt;BH1750.h&amp;gt;
0007
0008  static DHT dht(PIN_DHT22, DHT22);
0009  static BH1750 light_meter(BH1750_ADDR);
0010
0011  static int gas_history[GAS_SMOOTHING_SAMPLES];
0012  static int gas_history_index = 0;
0013  static bool gas_history_filled = false;
0014  static unsigned long boot_time = 0;
0015  static int gas_baseline = -1; // set once, right after warm-up ends
0016
0017  // MQ135 rail watchdog: a disconnected or shorted analog input still returns
0018  // an in-range ADC value (e.g. 0 when grounded), so a plain range check can&amp;#x27;t
0019  // catch it. Treat a reading pinned at either rail for longer than physically
0020  // plausible as a stuck/disconnected sensor rather than a real gas trend.
0021  static bool gas_in_rail = false;
0022  static unsigned long gas_rail_since = 0;
0023  static bool gas_was_stuck = false;
0024
0025  void sensors_init() {
0026    dht.begin();
0027    Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
0028    light_meter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE);
0029    analogReadResolution(12);
0030    boot_time = millis();
0031  }
0032
0033  static int gas_smoothed_average() {
0034    int count = gas_history_filled ? GAS_SMOOTHING_SAMPLES : gas_history_index;
0035    if (count == 0) return 0;
0036    long sum = 0;
0037    for (int i = 0; i &amp;lt; count; i++) sum += gas_history[i];
0038    return sum / count;
0039  }
0040
0041  int sensors_gas_baseline() {
0042    return gas_baseline;
0043  }
0044
0045  void sensors_read(Reading &amp;amp;out) {
0046    unsigned long now = millis();
0047
0048    // --- DHT22 ---
0049    // The library itself reports NaN on a failed/missing read, so that is the
0050    // failure signal — a steady room legitimately holds the same value for a
0051    // long time and must not be flagged invalid because of that.
0052    float t = dht.readTemperature();
0053    float h = dht.readHumidity();
0054    out.temp_valid = !isnan(t);
```

```
0055    out.humidity_valid = !isnan(h);
0056    out.temperature_c = t;
0057    out.humidity_pct = h;
0058
0059    // --- BH1750 ---
0060    // readLightLevel() returns a negative value on a failed read; a real dark
0061    // room legitimately reads 0 and must stay valid.
0062    float lux = light_meter.readLightLevel();
0063    out.light_valid = lux &gt;= 0;
0064    out.light_lux = lux;
0065
0066    // --- MQ135 ---
0067    int raw = analogRead(PIN_MQ135_AOUT);
0068
0069    bool at_rail = (raw &lt;= GAS_RAIL_LOW || raw &gt;= GAS_RAIL_HIGH);
0070    if (at_rail) {
0071      if (!gas_in_rail) {
0072        gas_in_rail = true;
0073        gas_rail_since = now;
0074      }
0075    } else {
0076      gas_in_rail = false;
0077    }
0078    bool gas_stuck = gas_in_rail &amp;&amp; (now - gas_rail_since &gt; GAS_RAIL_TIMEOUT_MS);
0079
0080    if (gas_was_stuck &amp;&amp; !gas_stuck) {
0081      // Just recovered from a stuck rail: the smoothing window still holds
0082      // rail samples, so discard it and require a full fresh window of
0083      // non-rail samples before the average/baseline can be trusted again.
0084      gas_history_index = 0;
0085      gas_history_filled = false;
0086    }
0087    gas_was_stuck = gas_stuck;
0088
0089    gas_history[gas_history_index] = raw;
0090    gas_history_index++;
0091    if (gas_history_index &gt;= GAS_SMOOTHING_SAMPLES) {
0092      gas_history_index = 0;
0093      gas_history_filled = true;
0094    }
0095    out.gas_adc_raw = raw;
0096    out.gas_adc_avg = gas_smoothed_average();
0097    out.gas_warming_up = (now - boot_time) &lt; MQ135_WARMUP_MS;
0098    out.gas_valid = !gas_stuck;
0099
0100    if (!out.gas_warming_up &amp;&amp; !gas_stuck &amp;&amp; gas_baseline &lt; 0 &amp;&amp; gas_history_filled) {
0101      gas_baseline = out.gas_adc_avg; // first stable average after warm-up
0102    }
0103  }
```

firmware/05_integrated_monitor/sensors.h

25 source lines

```
0001  // FreshAir Desk — sensor acquisition, validation, and smoothing
0002  #ifndef FRESHAIR_SENSORS_H
0003  #define FRESHAIR_SENSORS_H
0004
0005  #include &lt;Arduino.h&gt;
0006
0007  struct Reading {
0008    float temperature_c;
0009    float humidity_pct;
0010    float light_lux;
0011    int   gas_adc_raw;
0012    int   gas_adc_avg;
0013
0014    bool temp_valid;
0015    bool humidity_valid;
0016    bool light_valid;
0017    bool gas_valid;
0018    bool gas_warming_up;
0019  };
0020
0021  void sensors_init();
0022  void sensors_read(Reading &amp;out);
0023  int  sensors_gas_baseline(); // established once warm-up completes
0024
0025  #endif
```

setup.bat

92 source lines

```
0001  @echo off
0002  setlocal EnableExtensions
0003  title FreshAir Desk - Firmware Upload
0004
0005  echo.
0006  echo ============================================================
0007  echo   FreshAir Desk - Firmware Upload
0008  echo ============================================================
0009  echo.
0010
0011  set &quot;ARDUINO_CLI_DIR=%LOCALAPPDATA%\Programs\Arduino IDE\resources\app\lib\backend\resources&quot;
0012  if not exist &quot;%ARDUINO_CLI_DIR%\arduino-cli.exe&quot; set &quot;ARDUINO_CLI_DIR=%LOCALAPPDATA%\Programs\arduino-ide\resources\app\lib\backend\
      resources&quot;
0013  if not exist &quot;%ARDUINO_CLI_DIR%\arduino-cli.exe&quot; (
0014      echo [ERROR] Arduino IDE command-line tools were not found.
0015      echo Open Arduino IDE once, then run this file again.
0016      goto :failed
0017  )
0018  set &quot;ARDUINO_CLI=%ARDUINO_CLI_DIR%\arduino-cli.exe&quot;
0019
0020  echo [1/4] Checking ESP32 board package...
0021  &quot;%ARDUINO_CLI%&quot; core list 2&gt;nul | findstr /I /C:&quot;esp32:esp32&quot; &gt;nul
0022  if errorlevel 1 (
0023      echo Installing the ESP32 board package. This needs internet access...
0024      &quot;%ARDUINO_CLI%&quot; core update-index
0025      if errorlevel 1 goto :failed
0026      &quot;%ARDUINO_CLI%&quot; core install esp32:esp32
0027      if errorlevel 1 goto :failed
0028  )
0029
0030  echo [2/4] Checking sensor libraries...
0031  &quot;%ARDUINO_CLI%&quot; lib list 2&gt;nul | findstr /I /C:&quot;BH1750&quot; &gt;nul
0032  if errorlevel 1 (
0033      &quot;%ARDUINO_CLI%&quot; lib install &quot;BH1750&quot;
0034      if errorlevel 1 goto :failed
0035  )
0036  &quot;%ARDUINO_CLI%&quot; lib list 2&gt;nul | findstr /I /C:&quot;DHT sensor library&quot; &gt;nul
0037  if errorlevel 1 (
0038      &quot;%ARDUINO_CLI%&quot; lib install &quot;DHT sensor library&quot;
0039      if errorlevel 1 goto :failed
0040  )
0041  &quot;%ARDUINO_CLI%&quot; lib list 2&gt;nul | findstr /I /C:&quot;Adafruit Unified Sensor&quot; &gt;nul
0042  if errorlevel 1 (
0043      &quot;%ARDUINO_CLI%&quot; lib install &quot;Adafruit Unified Sensor&quot;
0044      if errorlevel 1 goto :failed
0045  )
0046
0047  echo [3/4] Finding the ESP32 USB connection...
0048  set &quot;ESP_PORT=&quot;
0049  set &quot;PORT_LIST=%TEMP%\freshair_board_list.txt&quot;
0050  set &quot;PORT_MATCH=%TEMP%\freshair_board_port.txt&quot;
0051  &quot;%ARDUINO_CLI%&quot; board list &gt; &quot;%PORT_LIST%&quot; 2&gt;&amp;1
0052  type &quot;%PORT_LIST%&quot;
0053  findstr /R /I /C:&quot;^COM[0-9][0-9]*&quot; &quot;%PORT_LIST%&quot; &gt; &quot;%PORT_MATCH%&quot;
```

```
for /f &quot;usebackq tokens=1&quot; %%P in (&quot;%PORT_MATCH%&quot;) do if not defined ESP_PORT set &quot;ESP_PORT=%%P&quot;
if not defined ESP_PORT (
    echo [ERROR] No COM port was found.
    echo Check that the ESP32 is connected by USB, then run this file again.
    goto :failed
)
echo ESP32 connection: %ESP_PORT%

echo [4/4] Building and uploading the dashboard program...
&quot;%ARDUINO_CLI%&quot; compile --fqbn esp32:esp32:esp32 &quot;%~dp0firmware\03_dashboard&quot;
if errorlevel 1 (
    echo.
    echo [ERROR] The program could not be built, so nothing was uploaded.
    echo Read the error shown above. If it says Windows could not start the compiler,
    echo restart the laptop and run this file once more.
    goto :failed
)

&quot;%ARDUINO_CLI%&quot; upload --fqbn esp32:esp32:esp32 --port %ESP_PORT% &quot;%~dp0firmware\03_dashboard&quot;
if errorlevel 1 (
    echo.
    echo [ERROR] The program built, but the upload failed.
    echo Close Arduino Serial Monitor if it is open, then try again.
    goto :failed
)

echo.
echo Upload completed. The ESP32 should join the iPhone hotspot.
echo Keep the laptop connected to the same hotspot and start ai_server\start.bat for AI insights.
echo The dashboard address is shown in the ESP32 Serial Monitor.
goto :done

:failed
echo.
echo Upload did not complete. No log file is needed.
:done
echo.
pause
endlocal
```

Upload_iPhone_Firmware.bat

2 source lines

```
0001  @echo off
0002  call &quot;%~dp0setup.bat&quot;
```